Are We Too Focused on Single Query Language? Investigating Text-to-SQL/SPARQL/Cypher Task Complexity via Fine-tuning Unbiased T5 Models.

Martin Vejvar[a,c], Yasutaka Fujimoto[b,c]

[a] *Yokohama National University, Graduate School of Engineering Science,*
[b] *Yokohama National University, Faculty of Engineering,*
[c] *Hodogaya, 79-1 Tokiwadai, Yokohama, 240-0067, Kanagawa, Japan*

Abstract

Neural text-to-query research remains very focused on SQL, often neglecting the benefits of graph query languages such as SPARQL and Cypher for systems involving highly interconnected data. We appeal that query language and data model choice should be considered as another parameter to evaluate during task-specific experiments. We demonstrate the significance of query language choice for text-to-query task by isolating the core factors that affect the performance of text-to-query models across these three distinct languages. We introduce Spider4SSC dataset, a multi-domain benchmark containing semantically equivalent SQL, SPARQL and Cypher queries and employ fine-tuned T5 models pre-trained on explicitly filtered plain text (unbiased-T5, uT5) to minimize pre-training data bias. Our analysis focuses on execution accuracy (`EX`) as a function of language complexity, pre-training exposure, schema information quality and multi-task learning effects. SPARQL is consistently the most challenging language for our uT5 models, showing the lowest average `EX`. We also observe a clear performance crossover between SQL and Cypher based on query complexity. SQL achieves higher accuracy for simple queries (58% vs 50%), while Cypher beats SQL on medium (38% vs 34%) and hard (14% vs 6%) queries, indicating Cypher is more expressive for complex questions. Furthermore, while SQL and SPARQL benefit from including relationships and data types in schema representation, Cypher achieves highest accuracy using a concise schema with only object and property names. Finally, jointly fine-tuning on all query languages improves Text-to-SQL performance (+2.47% `EX`), showing that exposure to graph data models benefits relational database reasoning.

Keywords: text-to-query, SQL, SPARQL, Cypher, T5, Database systems, query complexity, relational database, graph database, data models

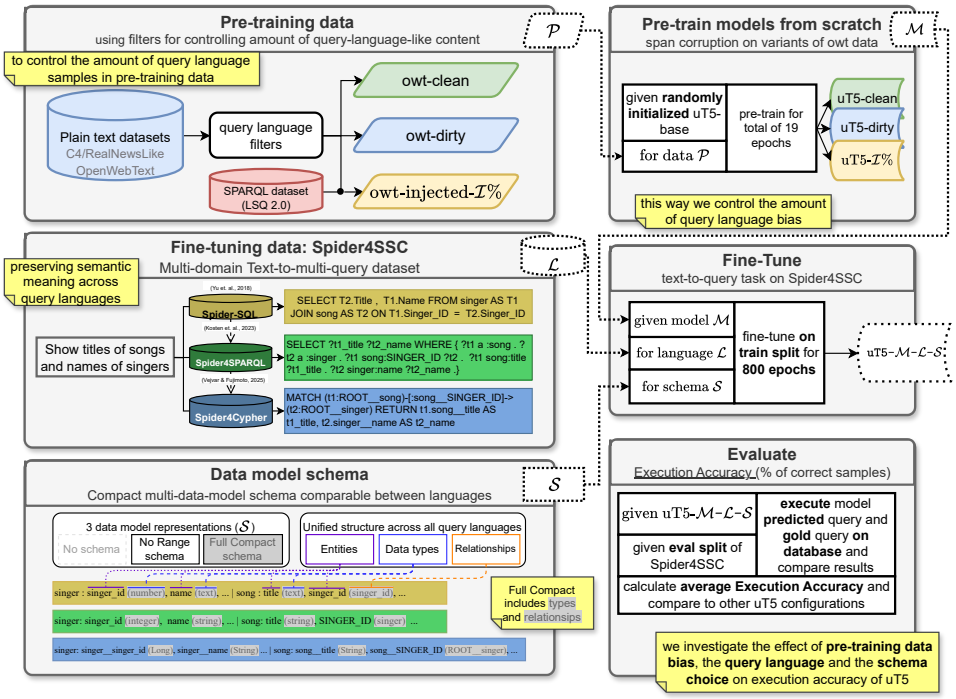

Figure 1: Overview of the experimental workflow. We filter pre-training data to control the amount of query-language-like content and use them to train T5 model with randomly initialised weights. Resulting unbiased-T5 (uT5) models are fine-tuned on our multi-domain Spider4SSC dataset containing semantically equivalent SQL, SPARQL, and Cypher queries, evaluated under consistent schema representations (No Schema, No Range, Full Compact) and compared by execution accuracy (`EX`).

1. Introduction

With the ever-growing need for data storage, efficient database systems are becoming increasingly more relevant. Language models (LMs) are emerging as powerful interfaces for users to express information needs in natural language, enabling text-to-query systems that automatically translate natural language into formal database queries such as SQL, SPARQL or Cypher.

While transformer-based LMs like T5 [1] have demonstrated significant performance gains, research is heavily dominated by text-to-SQL for relational databases [2].

However, relational database models often struggle with the efficiency required for complex and highly interconnected data [3, 4]. The necessity of using graph structures, such as resource description frameworks [5] or labelled property graphs [6], is evident to model complex knowledge [7, 8]. Translating natural language into SPARQL or Cypher naturally unlocks applications where graph data models are beneficial, extending beyond simple data retrieval to fields like semantic mapping and knowledge enrichment [9, 10, 11].

We can summarise our research paper with a simple question: Given a text-to-query task, how do we decide which query language (and thus data model) yields the best performance? In general, we appeal that the choice of query language should not be fixed, but rather treated as another parameter to evaluate in task-specific experiments.

Initial comparative studies [12] have suggested that certain query languages (e.g. SPARQL) may be intrinsically more challenging to learn for transformer models. However, this observation was neglecting coupled effects, such as the influence of pre-training data exposure or specific schema representation choices on language model performance. To our knowledge, no previous work has explicitly disentangled the effects of pre-training bias from the intrinsic structural complexity of query languages.

We propose a method that isolates these factors. We train an unbiased uT5 model (uT5) pretrained on corpora explicitly filtered to exclude query-language examples and further fine-tune on Spider4SSC, a multi-domain and multi-query-language dataset. We evaluate language complexity, pre-training data exposure and schema choices and how they independently affect transformer performance in text-to-query tasks across SQL, SPARQL, and Cypher. In this work we aim to increase awareness of the lack of diverse, comparative studies between query languages and data models in practical applications. Figure 1 shows the complete workflow of our benchmark. Follow these references for GitHub repositories with our pre-training code [13] and fine-tuning code [14]. In Section 1.1 we present five research questions that guide our experiments and the contributions of our paper.

1.1. Contributions

RQ1. Among SQL, SPARQL and Cypher, are certain query languages more challenging for language models to learn?

RQ2. Does bias in pre-training data affect the performance of language models on downstream text-to-query tasks *across SQL, SPARQL and Cypher*?

RQ3. Is there a correlation between the cognitive complexity of queries and the performance of the language model?

RQ4. What is the effect of schema information quality on text-to-query performance?

RQ5. Does multi-task training across SQL, SPARQL and Cypher enhance cross-language performance of language models?

2. Related Work

Translating natural language text to a formal query language (e.g. SQL) can be collectively grouped under the term *text-to-query*. Existing research generally focuses on the isolated evaluation of single-query languages, SQL [15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34], SPARQL [35, 36, 37, 38, 39, 40, 41, 42, 43] or Cypher [44, 45, 46, 47, 48, 49, 50] independently. Direct comparison between different query languages is limited and requires further study.

SM3-text-to-query benchmark. Sivasubramaniam et al. [12] address the gap by prompting large language models (LLMs) and evaluating their execution accuracy (`EX`) across SQL, MQL, SPARQL and Cypher query languages, using their synthetic multi-query-language SM3-Text-to-Query dataset. They conducted experiments with and without schema information and under zero- and few-shot in-context learning (ICL) settings. Their findings show that zero-shot performance favoured SQL (possibly due to complexity of the language or greater exposure in pre-training). This gap diminished after adding schema information or fine-tuning. However, the benchmark has notable limitations:

1. **No fine tuning**: Results report on general performance of large language models, which are few-shot instruction-tuned but not fine-tuned on the downstream task.
2. **Multi-factor effect**: Differences in performance between languages and data models have an unclear origin. The accuracy gain/loss may reflect pre-training bias rather than inherent language difficulty.

3. **Limited domain**: SM3 dataset queries all operate on a single synthetic database with Medical domain.
4. **Limited diversity**: The generation of query based on the template of the data set produces limited structural diversity, allowing fine-tuned models (e.g. T5) to achieve high performance with relatively little generalisation.

We fill the gaps by composing a multi-domain Spider4SSC dataset (Section 3.1) and decoupling the pre-training bias and schema choice from the query language complexity via pre-training dataset filtering (see Section 3.2).

2.1. Text-to-multi-query benchmarks

Our experiments fine-tune and evaluate the T5 model on the text-to-query task across multiple query languages. At the time of writing, SM3-Text-to-Query [12] is the only existing dataset explicitly designed for multi-query-language evaluation. However, due to complexity limitations discussed in Section 2.1.1, SM3 is not suitable for our fine-tuning setup. Therefore, we build our own multi-query-language dataset for SQL, SPARQL, and Cypher.

To achieve this, we first explore datasets that already contain partial multi-language correspondence, those seeded from a common data source and offering multiple semantically equivalent query examples for a single natural-language question. For SQL and SPARQL, this includes MIMICSQL [51] and its counterpart MIMIC-SPARQL [52], as well as the widely used Spider [15] and its SPARQL extension Spider4SPARQL [53]. Among these, we focus on the Spider family of datasets, as Spider and its variants [54, 55, 56] continue to dominate both historical and recent text-to-SQL research due to their complexity and domain diversity [21, 19, 20, 57, 26, 31, 33]. There is no equivalent text-to-Cypher dataset in this domain, which motivates the construction of our own extension, described in Section 2.1.2.

2.1.1. SM3-Text-to-Query

As previously mentioned, SM3-Text-to-Query [12] is a multimodal synthetic medical dataset. It contains 10K text/query pairs (6000 train, 2000 evaluation, and 2000 test samples), designed to benchmark language models across four distinct query languages and data representations (SPARQL, SQL, Cypher, and MQL). The questions are generated from 408 hand-crafted

templates using Synthea[1], with corresponding queries manually created for each language and double-checked by a second expert for correctness.

Limitations. SM3 is a useful baseline for evaluating text-to-query performance across multiple query languages. However, the single domain and reliance on templates leads to highly predictable query syntax, which can be easily memorised by fine-tuned models. As a result, it does not offer a reliable ground for assessing the model generalisation or the intrinsic difficulty of each query language.

2.1.2. From Spider to a multi-query-language dataset

As mentioned earlier, Spider [15] is a widely used large-scale text-to-SQL dataset composed of 200 relational (SQLite) databases and 5693 unique SQL queries, paired with 10181 natural-language questions. Recently, Kosten et al. [53] introduced Spider4SPARQL as a SPARQL counterpart to Spider. They materialised 166 of the original Spider SQLite databases into RDF graphs using the Ontop Virtual Knowledge Graph framework [58]. To ensure functional equivalence between the generated SQL and SPARQL queries, the authors employed SemQL, a context-free grammar that served as an intermediate translation layer. This process yielded 4721 SPARQL queries that are semantically equivalent to their corresponding SQL queries.

Spider4SSC. To further expand Spider and Spider4SPARQL into multi-query language, we develop Spider4SSC, a multi-domain text-to-query language dataset with 4525 unique (question, sql, sparql, cypher) samples where semantically matching queries are provided in SQL, SPARQL and Cypher (see Tab. 1). The dataset contains 159 distinct domain relational databases from Spider [15] and 159 equivalent graph databases. This allows cross-query-language benchmarking and fine-tuning of neural networks on text-to-query task. See Section 3.1 for brief methodology, [59] for detailed construction of Spider4SSC and [60] for dataset download.

2.2. Formal and Cognitive Complexity of Query Languages

Past research has focused heavily on theoretical computational complexity, specifically data complexity and the combined complexity of query languages in relational [61, 62] and graph databases [63]. Data complexity is

[1] `github.com/syntheticahealth/synthea-international`

Table 1: Counts of unique (question, sql, sparql, cypher) samples in Spider4SSC with equivalent execution results across all three query languages and the count of respective database files each split uses.

	Dev	**Train**	**Total**
Samples	608	3917	4525
Databases	20	139	159

related to the size of the database that is being queried, while combined complexity also takes the query parameters (such as length) into account. Our interest lies in the field of cognitive query complexity, i.e. a measure of how difficult it is to formulate and understand the semantics of the query within a given query language.

2.2.1. Towards cognitive complexity

In addition to data complexity, Moshel [64] defines **expression complexity** as a measure of the succinctness of a language. In other words, expression complexity measures how long (or compact) queries in this language must be to express various questions. It captures differences in how compactly different query languages can express the same question. By keeping the same database while changing the queries and query language, Moshel concludes that there is a clear gap between query languages in the ability to succinctly express different question types. As it is a measure of compactness, the expression complexity can also be changed simply by changing the syntax to shorter/longer representations.

Parametric complexity. Papadimitriou & Yannakakis [65] investigated how the parametrised size of the query and the number of variables affect tractability. They show that data complexity is exponentially dependent on the query length and that this exponential dependence becomes more pronounced as the expressive power of the query language increases. In other words, greater expressivity (and hence succinctness) tends to negatively impact the efficiency of data retrieval.

Cognitive complexity. Applying the parametric approach to expression complexity, we can estimate cognitive complexity as a function of the count of parameters (such as number of variables, nesting depth, number of JOIN statements, etc.) present in the query. By quantising query parameters (factors) and calculating weighted average of all the factors, we can estimate how

hard it is to construct and understand the given query. We define our cognitive complexity score in Section 3.4. In RQ3 (Section 4.3.3), we investigate the relationship between model performance and cognitive complexity score.

2.2.2. Empirical studies on query complexity

Empirical studies on relational databases are giving insight into complexity of query languages for humans. When participants are tasked with composing SQL queries based on given task and data models, empirical studies generally agree on expressivity being directly proportional to task performance [66, 67, 68]. However, Bowen et. al. [69] argue that this does not hold for bigger data models. They claim that increasing the size of the domain (data model) increases almost all aspects of task complexity. Given a database with large domain, 2 groups (of about 40 participants each) were presented with either parsimonious (sparse ontology) data model or a very expressive one (rich ontology). The researchers found that the general accuracy and confidence in answers was lower for the expressive model. Thus, Bowen et. al. conclude that there is a trade-off between parsimony and expressiveness of database models and having access to more detailed information is not always leading to higher accuracy on the given task. This leads directly to our experiments in RQ4 on schema representation (see Sections 2.3 and 4.3.4).

2.3. Role of Schema Representation

The impact of schema representation on the performance of the text-to-query model has been highlighted by several studies. Suhr et al. [70] demonstrated that the way schemas are serialised (compact or detailed) significantly affects text-to-SQL accuracy. More recently for Large Language Models, Gao et al. [71] introduced M-Schema, a semi-structured data model representation with incorporated data-types, which shows that a clearer depiction of the database structure guides LLM to higher accuracy. With respect to text-to-SPARQL query generation, Meyer et al. [72] show that the same SPARQL query task can yield vastly different model accuracies depending on whether the input is a full graph serialisation, a relevant sub-graph, a pure schema or a list of IRIs. For Property Graphs, Angles et al. [73] propose a simple and general schematic representation of graphs and compare to existing RDF and SQL paradigms. However, no prior work has systematically examined how schema verbosity and its information content interact with the complexity of different query languages under identical model architecture. We address this

gap in RQ4 by designing parallel schema serialisations for SQL, SPARQL, and Cypher, allowing fair evaluation across data models.

3. Methods

During our experiments, we need to decouple the effect of query language syntax and semantics on language model performance from the bias caused by query language sample imbalance in pre-training data. In Section 3.1, we briefly introduce our methodology on Spider4SSC dataset. Section 3.2 shows our filtering pipeline. In Section 3.3, we define our schema serialisation method, which makes the schemas comparable across the three different database models. Section 3.4 explains our cognitive query complexity calculation. Finally, Section 3.5 shows Execution Accuracy (`EX`), our main evaluation metric for all experiments.

3.1. Spider4SSC

Initially, we downloaded Spider4SPARQL dev and train split JSONs and materialised RDF knowledge graphs (KG) following the instructions on [53] GitHub[2]. Furthermore, we fixed and re-materialised 23 empty KGs and omitted 7 with no SQLite counterpart, resulting in a total of 159 valid and equivalent SQLite/RDF graph databases. Using our S2CLite rule-based SPARQL to Cypher parser (Figure. 2, details in [59]), we translated all parseable SPARQL queries in Spider4SPARQL into matching Cypher queries. Finally, downloading Spider and the respective SQLite databases[3], we assembled the **Spider4SSC** dataset from all executable and valid queries. To ensure that the queries are equivalent, all three queries must return matching results when executed over their respective database (using sqlite3[4] for SQL, RDF4j[5] for SPARQL and Neo4j[6] for Cypher). We include only question/query pairs that strictly adhere to this rule. Refer to Listing 1 for an example entry from Spider4SSC dev set. For a comparative study of the cognitive complexity of queries between the three languages, refer to §3.4. A detailed description of the dataset generation process and SPARQL-to-Cypher translation rules implemented in S2CLite is provided in our companion technical report [59].

[2] `https://github.com/ckosten/Spider4SPARQL`
[3] `https://yale-lily.github.io/spider`
[4] `https://docs.python.org/3.11/library/sqlite3.html`
[5] `https://rdf4j.org/`
[6] (Barrasa and Cowley, 2021) [74]

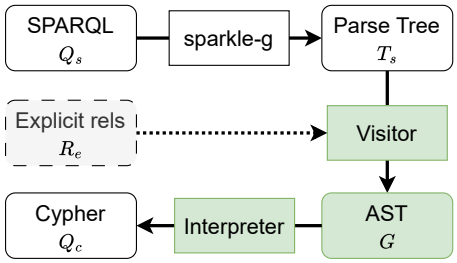

Figure 2: Overview of S2CLite parser. SPARQL query Q_s is transformed into SPARQL 1.1 Parse tree T_s using [75] grammar. Visitor traverses T_s and generates a Cypher pattern abstract syntax tree ($\mathcal{G}$), which is then used by Interpreter to build the final Cypher query Q_c. Visitor can optionally take a set of explicit relationship type names (R_e), enforcing them as relationships instead of node properties (see [59]).

Listing 1: Sample entry from Spider4SSC dataset.

```
{
    "db_id": "concert_singer",
    "question": "How many singers do we have?",
    "sql": "SELECT count(*) FROM singer",
    "sparql": "select (count( *) as ?aggregation_all)
               where { ?t1 a :singer . }",
    "cypher": "MATCH (t1:ROOT__singer)
               WITH COUNT(*) AS aggregation_all
               RETURN aggregation_all",
    "namespaces": [
        "concert", "stadium", "singer_in_concert",
        "singer", "XMLSchema"
    ]
}
```

3.1.1. Query alignment

Q3.3. Analyze potential selection bias in Spider4SSC construction. Spider4SSC is derived by filtering Spider4SPARQL from 4721 to 4525 examples based on cross-language execution equivalence. To demonstrate that the dataset is not biased toward easier-to-align queries, the authors could compare retained vs. removed samples in terms of hardness levels, query length, and complexity factor distributions. We will update the related Data in Brief co-submission to include these additional graphs

A. When comparing retained and removed samples in terms of hardness levels, query length, and complexity factor distributions, we see a clear bias toward easier-to-align queries in Spider4SSC compared to original Spider. We will update However, this does not influence comparisons across languages. We ensure that only matching queries remain, meaning that any more difficult queries are removed from all three query languages.

3.2. Filters for removing query languages

To ensure that the pre-training data does not contain examples of formal query languages (SQL, SPARQL, Cypher) that could bias model learning, we apply an explicit filtering procedure to both the RealNewsLike and OpenWebText datasets prior to pre-training.

By default, the RealNewsLike dataset already excludes documents containing curly brackets ("{", "}") to remove most programming-related text (see section 2.2 in Raffel et al. [1]). However, SQL does not use curly braces, while SPARQL and optionally Cypher does. This alone can cause bias towards SQL due to unfiltered queries being present. Thus, we apply our own filtering process to ensure a clean dataset with no samples of the three investigated query languages. This is especially significant for the OpenWebText corpus, which is scraped from Reddit and includes snippets from code discussions, tutorials, and documentation.

Filtering overview. For each text sample x, we sequentially apply two sets of filters:

- regular-expression filters $\mathcal{R}_j(x)$, capturing structured query syntax (e.g., `"SELECT ... FROM"`)
- keyword filters $\mathcal{K}_k(x)$, capturing residual fragments or standalone query terms

A text sample is flagged for removal if it's substring matches any regular expression or keyword filter:

$$r(x) = \bigvee_j \mathcal{R}_j(x) \tag{1}$$

$$k(x) = \bigvee_k \mathcal{K}_k(x) \tag{2}$$

$$y(x) = r(x) \vee k(x) \tag{3}$$

where $r(x)$ and $k(x)$ are Boolean indicators of whether any filter was activated. A sample x is retained only if $y(x) = \texttt{False}$ (i.e., it passes all filters).

3.2.1. Regular expression filters

We designed three case-insensitive regular expressions that identify characteristic query-language statements. See Table 2 for the detailed patterns.

Table 2: Regular expressions used for detecting query-language patterns.

Language	**Pattern**	**Matched substring**
SPARQL	`\bSELECT\b.*\bWHERE\b`	`SELECT` followed by `WHERE`
SQL	`\bSELECT\b.*\bFROM\b`	`SELECT` followed by `FROM`
Cypher	`\bMATCH\b.*\bRETURN\b`	`MATCH` followed by `RETURN`

3.2.2. Keyword filters

To capture residual fragments or incomplete queries not caught by regular expressions (e.g., code comments, schema listings, or tutorial snippets), we apply a set of case-insensitive keyword filters. See Listing 2 for all keywords.

Listing 2: Keywords used to trigger query filtering.

```
["SELECT", "MATCH", "WHERE", "JOIN", "OPTIONAL", "FILTER",
 "RETURN", "INSERT", "DELETE", "SQL", "SPARQL", "CYPHER"]
```

Implementation. The filtering is applied sequentially. First, using the regular expressions, then the keyword list over each text sample. If a match is detected at any stage, the document is discarded. The full implementation is provided in Appendix C.

3.3. Schema representation

As previously mentioned in Section 2, we adopt a schema serialisation approach inspired by the work of Suhr et al. [70]. We focus on the fair comparison of three query languages. In this light, our formulation generalises the approach of Suhr et al. to three database paradigms: relational (SQL), Resource Description Framework (SPARQL), and property graph (Cypher). Furthermore, we define two schema variants: *No Range* and *Full Compact*. Each produces a linearised textual representation of the database schema that can be concatenated with a natural-language query as model input.

Definition. Let database be denoted as $D = (E, R)$, where $E = \{T_1, \ldots, T_n\}$ is the set of entities (tables, resource types, or node labels), and R the set of relationships (foreign keys, edges, or properties). Each entity T_i with name $\overline{T}_i$ consists of a sequence of attributes $\{c_{i,0}, c_{i,1}, \ldots, c_{i,|T_i|}\}$.

The schema serialisation $\mathcal{S}$ is defined as the concatenation of all entity serialisations t_i:

$$\mathcal{S} = t_0 + t_1 + \cdots + t_{|E|} \tag{4}$$

where each entity serialisation t_i is defined as:

$$t_i = | + \overline{T}_i + c_{i,0} + c_{i,1} + \cdots + c_{i,|T_i|} \tag{5}$$

Below we present the two schema variants, which differ in the internal representation of $c_{i,j}$.

3.3.1. No Range

The *No Range* variant captures only the names of entities and their attributes, excluding any data type or relationship information. In other words, each $c_{i,j}$ consists only of the column or property name:

$$c_{i,j} = C_{i,j} \tag{6}$$

3.3.2. Full Compact

The *Full Compact* variant extends the representation by including both attribute types and relationship information:

$$c_{i,j} = C_{i,j}\,(CT_{i,j}) \tag{7}$$

In eq. (7), $CT_{i,j}$ denotes either of:

a) a primitive data type of the attribute (e.g., *integer*, *string*, *boolean*)
b) a reference to another entity when the attribute represents a foreign key or graph edge (R).

Including both data type and/or related entity name gives model further information about connectivity between entities. In theory, we expect improvement in using this schema, with the trade-off of longer context window requirements.

3.3.3. Cross-database generalisation

We generalise the serialisation to support three distinct database representations:

- **SQL:** T_i represents a relational table, $C_{i,j}$ is a column name and $CT_{i,j}$ is either a data type or the name of a referenced column in a foreign table.
- **SPARQL (RDF):** T_i corresponds to a resource type (`r rdf:type singer`), and each $C_{i,j}$ is a property of that resource. If a property links to another resource, its target resource type replaces the literal data type.
- **Cypher (Property Graph):** T_i is a node label (e.g., `ROOT__singer`), and attributes $C_{i,j}$ represent node properties (e.g. `age`) or relationships (e.g. `CONCERT_ID`) to other nodes. In case of property $CT_{i,j}$ is data type, while in case of relationship $CT_{i,j}$ is the label of the related node.

3.3.4. Final serialisation format

The final serialised input to the model is defined as the following string:

$$\texttt{"}(\mathcal{L})\ q \mid D_{nm} \mid \mathcal{S}\texttt{"} \tag{8}$$

where $\mathcal{L}$ denotes the query language identifier (`sql`, `sparql`, or `cypher`), q is the natural-language question, D_{nm} is the database name, and $\mathcal{S}$ is the serialised schema as defined above.

This way we represent heterogeneous database structures consistently, enabling cross-language and cross-database generalisation of the text-to-query approach in our experiments.

For detailed examples of both *No Range* and *Full Compact* serialisations across SQL, SPARQL, and Cypher, see Appendix B.

3.4. Query Complexity

Inspired by Sivasubramaniam et al. [76], we define the theoretical cognitive complexity of a query q as a score $C(g)$, which integrates multiple structural and syntactical aspects of the query. Mathematically, the score is given as a weighted average of discrete complexity factors:

$$C(g) = \frac{\sum_{i \in F} w_i \cdot f_i(q)}{\sum_{i \in F} w_i}, \tag{9}$$

where $F = \{\text{distinct}, \text{tokens}, \text{keywords}, \text{joins}, \text{nesting}, \text{filters}, \text{aggregations}, \text{sorting}, \text{projections}\}$.

Parameters:. The complexity calculation utilises predefined weights w_i associated with each complexity factor f_i. These weights reflect the relative impact or importance of each characteristic in contributing to the overall complexity score. See tab. 3 for complete list of weight values for each factor. Individual complexity factors $f_i(q)$ are described in more detail in Appendix A.

Table 3: Overview of used weights for each complexity factor which are used to calculate the weighted complexity score in eq. (9). See Appendix §Appendix A for detailed complexity factor calculations.

Complexity Factor	**Symbol**	**Weight** (w_i)
Distinct Variables	$f_{distinct}$	1
Tokens	f_{tokens}	0.5
Keywords	$f_{keywords}$	1
Joins/Traversals	f_{joins}	2
Nesting Depth	$f_{nesting}$	2
Filters	$f_{filters}$	1.5
Aggregations	$f_{aggregations}$	1.5
Sorting/Limiting	$f_{sorting}$	1
Projections	$f_{projections}$	1

Fairness. As `EX` relies purely on execution results, it is not influenced by syntactic differences between query languages and only evaluates semantic accuracy of the generated queries to the gold queries.

3.5. Execution Accuracy

We adopt the **Test Suite Execution Accuracy** metric proposed by Zhong et al. [77], which currently serves as the official evaluation standard for the Spider leaderboard. Unlike exact set matching of query strings, execution accuracy avoids false negatives caused by syntactically different but semantically equivalent queries.

Let Y be the set of ground-truth queries and $\hat{Y}$ be the set of predicted queries. For a given pair $(y, \hat{y})$, let $V(y, D)$ denote the execution result of query y on database D. The execution accuracy is defined as:

$$\texttt{EX} = \frac{1}{N}\sum_{n=1}^{N}\mathbb{I}(V(y_n, D), V(\hat{y}_n, D)) \tag{10}$$

where $\mathbb{I}(\cdot)$ is an indicator function that returns 1 if the execution results are denotationally equivalent, and 0 otherwise.

3.6. Extended Execution Framework

The original implementation by Zhong et al. [77] relies on a local SQLite environment. To support our multi-language benchmark (SQL, SPARQL, and Cypher), we extended the evaluation code to interface with RDF4j (for SPARQL) and Neo4j (for Cypher) endpoints via asynchronous execution. While preserving the core logic of the original test suite, specifically its robust handling of column permutations and bag semantics, we introduced a unified normalisation pipeline to ensure cross-language compatibility.

Query Pre-processing. Prior to execution, all predicted queries undergo syntactic normalisation to correct tokenization artifacts common in sequence-to-sequence generation. This includes:

- **Token cleanup:** Removal of decoder-specific tokens (e.g., `</s>`, `<pad>`). Replacing doubled tokens (`{{` and `}}`) with single token.
- **Operator repair:** Removing space between split operators. For example, `"> ="` becomes `">="`.
- **Distinct removal:** The `DISTINCT` keyword is stripped from all queries to strictly enforce bag semantics, ensuring that duplicate rows are accounted for in the evaluation.

Result Post-processing and normalisation. A major challenge in multi-language evaluation is the heterogeneity of execution outputs. SQLite returns standard Python primitives, whereas RDF4j returns XML Schema Datatypes (XSD) and Neo4j returns Java-based driver types. To utilise the standard comparison metric, we implemented a normalisation layer that standardises these outputs into a uniform tuple format.

1. **Type Unification:** RDF literals and Neo4j types are mapped to native Python types (`int`, `float`, `bool`, `str`). For example, the normalisation layer converts `"5.5"^^xsd:decimal` to a Python `float`.

2. **Numeric Smoothing:** To account for floating-point precision differences between database engines, all decimals are rounded to a fixed precision (1 decimal place).
3. **Null standardisation:** SQL `NULL`, SPARQL unbound variables, and Neo4j `null` are universally mapped to Python `None`.

Denotation Equivalence. Once normalised, the result sets G (Gold) and P (Predicted) are compared using the permutation-invariant logic defined in Zhong et al. [77]. Equivalence is established if there exists a column permutation π such that the multiset of rows in G is identical to the multiset of rows in P (applying π to the columns of P). If the gold query contains an `ORDER BY` clause, the row order must also match exactly. For pseudocode on execution equivalence, refer to Appendix F.1.

4. Experiments and Evaluation

We compare performance of custom-pre-trained unbiased T5-base (uT5) model on a downstream task of text-to-query given a target query language (SQL, SPARQL, Cypher), respective database system (relational database/-knowledge graph) and its schema serialisation (see Sec. 3.3). See figure 3 for full experimental evaluation pipeline.

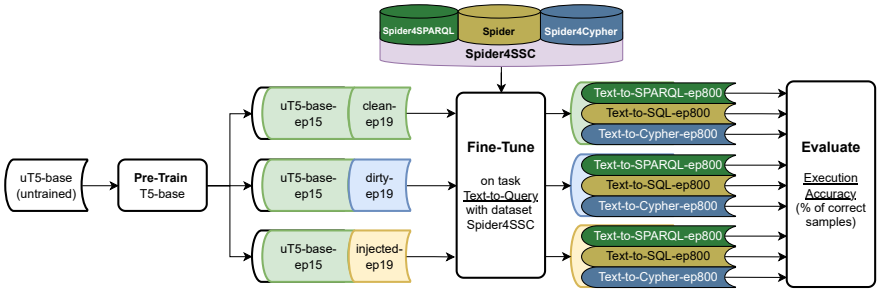

Figure 3: Flow of the full benchmark methodology. Firstly, we pre-train a randomly initialized t5-base model on 15 epochs of OpenWebText and C4/RealNewsLike, and further pre-training for 4 more epochs on three distinct data-splits of plain-text dataset (OpenWebText). The clean is free of any query languages, the dirty is unfiltered and kept as the original and injected has part of it replaced by pure SPARQL queries. Secondly, we fine-tune for text-to-query task on Spider4SSC dataset, which has 3917 questions and 3 semantically equivalent queries (SPARQL, SQL and Cypher) for each question.

4.1. Pre-training Setup

We follow the pre-training setup of the original T5 paper [1] using our custom datasets (see Section 4.1.1). We specifically focus on T5-base[7], which has 220 million trainable parameters and was originally pre-trained on the full C4/en[8] dataset, an English only text corpus web-scraped by Common Crawl[9]. Due to limited computing and storage resources, we use two relatively smaller datasets in our pre-training, (1) OpenWebText[10] [78] and (2) C4/RealNewsLike[11]. As the aim of this research is to provide a fair comparison between the performance of text-to-query models (not to achieve SOTA performance), we find these two datasets sufficient. See Section 4.1.1 for details on the filtering and preparation of pre-training data sets.

Task. The pre-training task is Span Corruption, which is to randomly sample and drop 15% of tokens and then replace consecutive spans of dropped tokens by a single sentinel token. The target is to reconstruct all of the dropped-out spans, delimited by the respective sentinel tokens.

Pre-training parameters. In baseline setup, Raffel et al. [1] pre-train for 2^{19} steps (with batch size of 128 sequences), which amounts to $2^{35} \approx 34B$ tokens of the pre-training dataset. We replicate this approach, using the initial learning rate of 0.01 with an inverse square root learning rate schedule $(1/\sqrt{max(n,k)})$ and $k = 10^4$ warm-up steps. However, because we are working with smaller dataset than full C4, we reset the learning rate several times during pre-training to avoid getting stuck at local minima.

4.1.1. Pre-training datasets and models

During our experiments, we do not use the already pre-trained variants of google-t5/t5-base models, but pre-train uninitialized T5 models from scratch (unbiased-T5, **uT5**). To control the bias caused by the imbalance of query language samples in pre-training data, we modify existing large plain-text datasets, specifically C4/RealNewsLike [1] and OpenWebText [78]. The filtering method we use is explained in detail in Section 3.2. Figure 4 shows the overview of the generated datasets.

[7] `https://huggingface.co/google-t5/t5-base`
[8] `https://huggingface.co/datasets/allenai/c4/tree/main/en`
[9] `https://commoncrawl.org/`
[10] `https://huggingface.co/datasets/Skylion007/openwebtext`
[11] `https://huggingface.co/datasets/allenai/c4/tree/main/realnewslike`

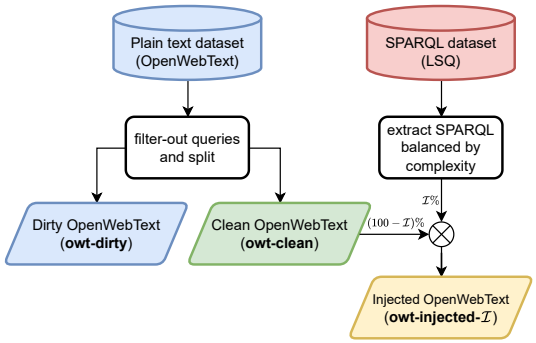

Figure 4: Overview of how the 3 types of pre-training datasets were generated. The baseline is OpenWebText dataset, which is split into equivalent size clean (filtered to not contain any queries) and dirty (unfiltered) parts. Finally, the injected split is generated by replacing $\mathcal{I}\%$ of the clean data by random sample of SPARQL queries from the LSQ2.0 dataset.

uT5-base-ep15. We first seed all our models with a t5-base model, randomly initialised and pre-trained from scratch for 15 epochs on the combination of OpenWebText and C4/RealNewsLike diluted by our query filters (see Section 3.2) to remove any samples with SQL, SPARQL or Cypher structures. This gives the model basic understanding and comprehension of the English language without being biased towards any specific query language.

Pre-trained model variants. Using our uT5-base-ep15 checkpoint, we further pre-train for 4 more epochs, branching into 3 (resp. 4) variants of uT5-base-ep19 models. Based on which pre-training dataset is used (see Section 3.2), we get:

1. **uT5-clean-ep19**: Trained on owt-clean where data has no query language bias (filtered by Sec. 3.2)
2. **uT5-dirty-ep19**: Trained on owt-dirty where data is more general and indicative of typical sample distribution in the OpenWebText dataset (with reduced size).
3. **uT5-injected-ep19-$\mathcal{I}\%$**: Trained on owt-injected. Data has artificially induced bias by replacing $\mathcal{I}\%$ of clean dataset with pure SPARQL queries from the large SPARQL query dataset LSQ 2.0 [79]. Specifically, we generate and pre-train on datasets with 10% and 30% SPARQL injection.

4.2. Fine-tuning Setup

For all downstream text-to-query task, we always train on the train set of Spider4SSC (Section 3.1) for 800 epochs with constant learning rate of 0.0001 and effective mini-batch size of 4096 using the Adagrad optimizer. After finishing a fine-tuning run, we automatically pick the best performing checkpoint (based on `EX`) for evaluation on the eval subset of Spider4SSC.

4.3. Experiments

In terms of answering the research questions (RQs) posed in Section 1.1, we perform the following experiments. In all experiments, we calculate average execution accuracy (`EX`) values from the 608 sample eval subset of the Spider4SSC text-to-multi-query dataset (see 2.1.2).

4.3.1. RQ1 (query language difficulty)

Given Spider4SSC dataset to fine-tune base models for text-to-query task, we measure the effect of query language choice on the execution accuracy (`EX`) of fine-tuned models. To fairly compare performance across different query languages, we first eliminate the effect of query language samples in pre-training data by training the uT5-clean, which we trained on the query-free version of OpenWebText dataset (owt-clean, see Section 4.1.1). Subsequently, we fine-tune uT5-clean on all three Spider4SSC subsets using the Full Schema database serialisation. Observe the first column of Table 4 for the results.

Discussion. SQL clearly achieves the highest execution accuracy (37.2%), with Cypher being only 2% lower with 35.2%. This suggests that **SQL and Cypher are similarly difficult to learn for T5 models, with SQL being slightly more succinct overall** (i.e. better at expressing questions). On the other hand, SPARQL shows a significant drop with `EX` of only 22.5%. This supports our hypothesis and the previous findings of Sivasubramaniam et al [12] that **SPARQL poses the greatest challenge for T5 models** when compared to SQL and Cypher. We follow up with RQ2 to verify the effect of bias introduced by the pre-training data.

4.3.2. RQ2 (pre-training data effect)

To discover the effect of pre-training data content on the performance across query languages, we fine-tune each pre-trained variant of uT5-base individually on each query language using Full Compact schema serialisation (See 3.3). Table 4 shows the complete evaluation results on each respective

subset of Spider4SSC with average Execution Accuracy (EX) percentages. Overall, we see highest accuracy for text-to-SQL-fine tuned models, followed by text-to-Cypher in second place and Text-to-SPARQL indicating lowest accuracy. SQL shows 2.7% EX increase for base of uT5-dirty over other base models. Cypher indicates highest performance for uT5-clean base with EX of 35.2%. SPARQL performs best for base models injected artificially with SPARQL queries (uT5-injected). We observe steady improvement over clean base (1.2% and 1.5% with 10% SPARQL injection and 30% SPARQL injection respectively). When calculating the standard deviation between EX of all base models, we arrive to 0.6% points, 0.9% points and 1.2% points for SPARQL, SQL and Cypher respectively.

Table 4: Execution accuracy (EX) percentages on evaluation set of Spider4SSC using four variants of pre-trained uT5 models (clean, dirty, injected with 10% SPARQL and injected with 30% SPARQL) and three query languages of Spider4SSC (SPARQL, SQL, Cypher) with average EX percentages. With respect to RQ1, we can see significant differences between languages, where SQL universally shows highest EX, while SPARQL has to lowest performance overall. With respect to RQ1, the marginal differences between column averages indicate only small effect of pre-training data on the final performance after fine-tuning.

EX (%)	uT5-clean	uT5-dirty	uT5-injected (10%)	uT5-injected (30%)	*Average*
sparql	22.5	23.2	23.7	**24.0**	23.4
sql	37.2	**39.0**	37.3	37.3	37.7
cypher	**35.2**	32.7	33.9	32.7	33.6
Average	31.6	31.6	31.6	31.4	31.6

Discussion. Overall, our experiments show that the choice of pre-training data has relatively small effect on fine-tuned model performance across all languages. This indicates that **choice of fine-tuning data is more important than the composition of pre-training data**. Nonetheless, we can see a clear bias towards SQL performance when pre-training on dirty dataset (sql, uT5-dirty). We also observe a similar bias towards higher SPARQL performance as we increase the injection ratio of SPARQL queries into the pre-training set (sparql, uT5-injected-10 and uT5-injected-30). On the other hand, Cypher shows best performance for clean dataset, which corresponds to the limited amount of samples for Cypher in both dirty and injected variations of pre-training set. It also suggests that **introducing examples of other query languages into the pre-training set generally hurts the performance of text-to-Cypher task**.

4.3.3. RQ3 (query complexity)

In previous experiments, we observed a significant `EX` difference between query languages, showing SQL as the best, Cypher as the middle, and SPARQL as the worst. To verify if this trend holds mathematically on a granular level, we define a complexity score that reflects how difficult a query is to understand (see section 3.4) and group the queries in the evaluation dataset by hardness.

Hardness level. Yu et. al [15] categorise four distinct levels of hardness for the SQL queries within Spider dataset. In the following experiments, we borrow these hardness levels to group queries into four categories (easy, medium, hard and extra) and evaluate their average complexity. We generalise the grouping so that the queries of SPARQL and Cypher subset of Spider4SSC evaluation dataset are assigned to the same hardness level group as their respective SQL query. This way we ensure that there is an equivalent subset of SQL, SPARQL and Cypher queries in each of the hardness level groups.

Comparing `EX` against inverse query complexity. Given individual query predictions p by uT5-clean (fine-tuned on the respective language) and their gold equivalents g, we calculate average execution accuracy values for each hardness level subset (see tab. 5a). Similarly, we calculate the average complexity score $C(g)$ of each hardness subset using eq. (9) and report the inverse value of the complexity score $\frac{1}{C(g)}$ (see tab. 5b). In Figure 5 we also provide a visual comparison of the two metrics.

Discussion. If we look at each language separately, we can see a monotonous downward trend for both execution accuracy and inverse complexity as the hardness level increases. Similarly to results from RQ1 and RQ2, SPARQL shows the lowest performance and SQL and Cypher have almost equal performance, as well as inverse complexity. However, given the individual hardness level groups, **SQL outperforms Cypher on easy queries**, while Cypher outperforms SQL for medium and hard queries. This suggests that **harder questions are easier to express with Cypher than with SQL**. Notice also that the swap from SQL to Cypher in top `EX` between easy and medium/hard queries also corresponds the same swap happening in the inverse complexity score. There is a clear inverse proportionality between model execution accuracy and cognitive query complexity. Finally, no single model was able to successfully predict any of the extra hard queries.

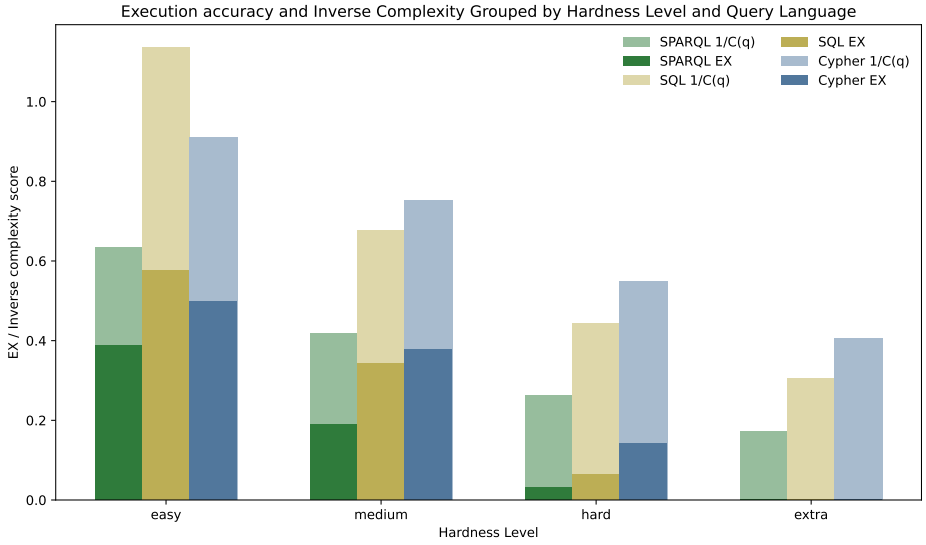

Figure 5: Bar plot overlapping average execution accuracy of uT5-clean predicted queries (EX(p)) and average inverse complexity score of respective gold queries ($\frac{1}{C(g)}$) given four sets of queries grouped by their hardness level as defined for SQL queries by Yu et al. [15].

4.3.4. RQ4 (schema effect)

In this experiment, we evaluate the effect of including schema and how the amount of information in schema affect model execution accuracy (EX) across SQL, SPARQL, and Cypher when using the `uT5-clean` model. Three schema configurations were tested: *No Schema*, *No Range*, and *Full Compact*. The *No Schema* setup provides only the question text, while the other two include increasing levels of detail about the database model (see Section 3.3). Table 6 summarises the execution accuracy across schema types and the accuracy gains of using a schema for all three query languages.

Table 5: Execution accuracy (EX) and inverse complexity ($\frac{1}{C(g)}$) across Spider hardness level query groups.

(a) EX by hardness level

	Hardness level (Spider)				
EX	easy	medium	hard	extra	*Average*
sparql	0.39	0.19	0.03	0.00	0.15
sql	0.58	0.34	0.06	0.00	0.25
cypher	0.50	0.38	0.14	0.00	0.25
Average	0.49	0.30	0.08	0.00	0.22

(b) Inverse complexity by hardness level.

	Hardness level (Spider)				
$\frac{1}{C(g)}$	easy	medium	hard	extra	*Average*
sparql	0.63	0.42	0.26	0.17	0.37
sql	1.13	0.68	0.44	0.30	0.64
cypher	0.91	0.75	0.55	0.41	0.65
Average	0.89	0.62	0.42	0.29	0.55

Discussion. As expected, the inclusion of the schema substantially improves performance across all query languages. Appending schema after the question improves SPARQL and Cypher accuracy by approximately 4.4 times, while SQL improves only ca 3 times. On the other hand, SQL achieves the highest zero-schema accuracy (9.38%), suggesting that **SQL is easiest to interpret from natural language pre-training data**. This is an interesting parallel to the higher performance of SQL when it comes to easy queries (RQ3, Section 4.3.3). SPARQL continues to perform the worst overall as in the previous research questions. Schema-wise, SPARQL and SQL perform best with the Full compact schema. However, **Cypher achieves the highest accuracy overall under the *No Range* schema (38.82%) and underperforms with the Full compact schema (35.20%)**. This is irregular to the other results and we hypothesise two possible reasons:

a) Information density. It is possible that the No Range schema already contains all the necessary information for Cypher to infer the data structure and including data type and relationship details increases schema redundancy and model confusion.
b) Context truncation. Due to the default maximum 512 token context-length limitation of the T5 model, input samples that are longer are cropped and the model is forced to work with incomplete schema of the database , reducing the performance. When evaluating the token truncation counts (see Tab. 8), we see average 14.5% truncation of cypher samples with Full Compact schema, while No Range is less affected (7.7% on average).

In Section 4.3.6, we analyse both possible eventualities by increasing the maximum input token length of our uT5-clean model 1024 and 2048 samples and fine-tuning from scratch .

Table 6: Execution accuracy (`EX`) and schema gain across schema configurations.

(a) `EX` (%) by schema configuration

	No Schema	No Range	Full Compact	*Average*
sparql	4.11	17.76	**22.37**	14.75
sql	9.38	36.84	**37.01**	**27.74**
cypher	6.58	**38.82**	35.20	26.86
Average	6.69	31.14	31.52	23.12

(b) `EX` gain of using schema

language	absolute (%)	relative
sparql	18.26	4.44
sql	27.63	2.95
cypher	28.62	4.35

4.3.5. RQ5 (multi-query-language fine-tuning effect)

In our final experiment, we explore whether joint fine-tuning of one model on multiple query languages at once (SQL, SPARQL, Cypher) yields better generalisation compared to fine-tuning with a single query language. We train `uT5-clean` model on the combined Spider4SSC dataset containing samples from all three query languages (with input prefix indicating which query language should be produced) and compare its performance with individually fine-tuned models from Section 4.3.4. The results in Table 7 show the absolute execution accuracy (`EX`) for each schema configuration (Tab. 7a) and the change in `EX` compared to single-language models (Tab. 7b). The same comparison is also shown in Figure 6 for clarity. Token truncation was confirmed as the primary cause for the Full Compact schema's underperformance at 512 tokens.

Once truncation is eliminated (at 2048 tokens), the performance of No Range (40.63

This suggests that the extra information in Full Compact (i.e., data types) is not just "unnecessary" but constitutes "information noise" that slightly hinders the model's performance.

Evaluation. From the average change in accuracy (Δ`EX` in Tab. 7b) we observe that joint fine-tuning with SQL, SPARQL, and Cypher did not yield a universal improvement. Language-wise, SQL benefits the most from multi-language fine-tuning, improving by 2.47% points, while Cypher experiences the largest performance drop (-2.08% points). SPARQL remains mostly unaffected (-0.38% points). Schema-wise, SQL improves across all schema variants, with the strongest gain in the *No Range* (3.45% points) and *Full Compact* (2.47% points) setups. In contrast, Cypher degrades in schema configurations (*No Range* and *Full Compact*, both -3.13% points). SPARQL remains stable across configurations, with a minor gain under *Full Compact* (+0.99% points).

Discussion. Overall, multi-query-language fine-tuning improves execution accuracy only for text-to-SQL. For SPARQL and Cypher the performance drops. This suggests that **exposure to other structured query languages improves the model's understanding of relational database schema**. Furthermore, we observe similar behaviour as in RQ4 (Section 4.3.4), where No Range schema over-performs Full Compact schema accuracy. In case of multi-query-language model not only for Cypher but also for

Table 7: Execution accuracy (EX) of uT5-clean model fine-tuned jointly on all three query languages in Spider4SSC, compared with language-specific fine-tuned model results (Table 6a).

(a) EX (%) by schema configuration for Multi-query-language-trained model.

EX (%)	No Schema	No Range	Full Compact	*Average*
sparql	4.11	15.63	**23.36**	14.36
sql	10.86	**40.30**	39.47	**30.21**
cypher	6.58	**35.69**	32.07	24.78
Average	7.18	30.54	**31.63**	23.12

(b) ΔEX compared to Tab 6a values (Single-query-language uT5-clean).

Δ EX (%)	No Schema	No Range	Full Compact	*Average*
sparql	0.00	-2.14	**0.99**	-0.38
sql	1.48	**3.45**	2.47	**2.47**
cypher	0.00	**-3.13**	-3.13	-2.08
Average	**0.49**	-0.60	**0.11**	0.00

SQL. See Section 4.3.6 for further details on this phenomenon.

4.3.6. Ablation: Input Truncation effect

As discovered in Sections 4.3.4 and 4.3.5, models with No Range schema perform better than with Full Compact schema (EX% of 38.82% vs 35.2% respectively) for Cypher (and SQL in case of 4.3.5). This does not conform with the other experiments, where Full Compact schema always comes on top. We propose that this discrepancy is caused by the truncation of inputs due to the limited context length of the T5-base model.

Setup. To test our hypothesis, we fine-tune and evaluate two additional models on the text-to-Cypher task with Spider4SSC dataset. One with a context length maximum of 1024 input tokens and one with further increase to a maximum of 2048 input context length. As the available context length increases, the average truncation of inputs falls, giving models more complete information about the database schema. Table 8 shows an overview of our ablation results. See Table E.10 for the average input token count and truncation statistics across the Spider4SSC dataset.

Table 8: Ablation of model context length for text-to-cypher with Full Compact and No Range schema settings, comparing the truncation ratio with model execution accuracy on the eval dataset of Spider4SSC. In general, we observe higher performance for No Range schema. Full Schema accuracy increases steadily as context length rises. No Range Schema performance drops slightly for 1024 tokens, while rising significantly for 2048 tokens.

Context length (tokens)	Schema Type	Truncation Ratio (%)	Execution Accuracy (%)
512	Full Compact	14.52	35.20
	No Range	7.71	**38.82**
1024	Full Compact	3.02	37.83
	No Range	0.91	**38.16**
2048	Full Compact	0.52	39.47
	No Range	0.18	**40.63**

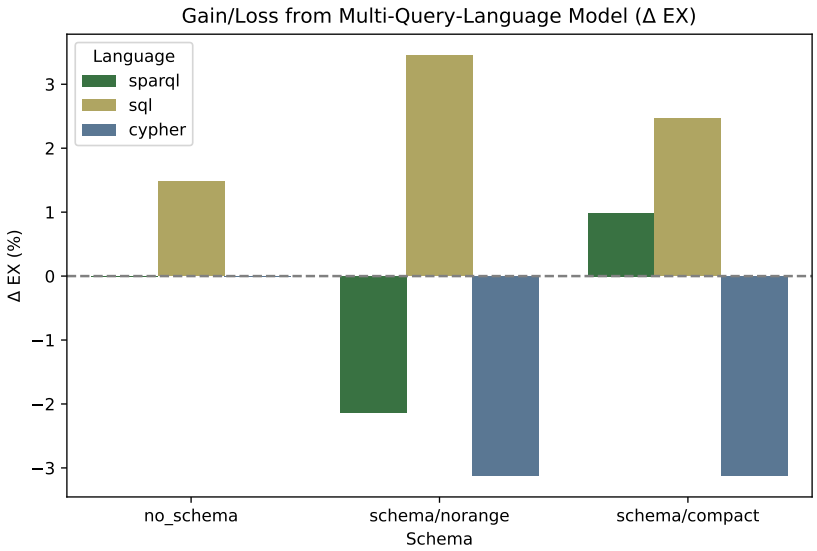

Figure 6: Bar plot of Gain or Reduction in execution accuracy % of uT5-clean ($\Delta\texttt{EX}(p)$) when trained in Multi-Query-language scenario compared to uT5-clean fine-tuned individually on each language. Values are averaged from the eval set of Spider4SSC.

Discussion. Even with increased context length, No Range schema still maintains a lead in Execution accuracy over Full schema. Nevertheless, Full schema shows a steady rise in accuracy as the truncation ratio falls. For the No Range Schema, we see a slight reduction of accuracy for context length of 1024 and a significant increase when context length is 2048. We conclude that **text-to-Cypher execution accuracy benefits from** both **higher information density** and the **lower context truncation of the No Range schema**. Thus, No Range schema is a better choice than Full Compact schema in case of Cypher.

5. Discussion and Conclusions

We presented a multi-domain benchmark for text-to-query models across SQL, SPARQL and Cypher. Using the Spider4SSC dataset, we isolated several factors affecting the performance of the T5 model. In below sections, we discuss the main findings with respect to the posed research questions and the main practical points of our experiments.

RQ1: Language Difficulty. **Certain query languages are indeed more challenging for the T5 model to learn**. Namely, SPARQL performed worst, with an average `EX` of 22.5%, compared to SQL's 37.2% and Cypher's 35.2% (Table 4).

RQ2: Pre-training Bias. We observed a small bias towards SQL in uT5-dirty (due to SQL in pre-training data) and bias toward SPARQL in uT5-injected (due to injection of SPARQL queries), the average performance of the models was stable at ca 31.6% `EX`, regardless of whether the uT5-clean, uT5-dirty, or uT5-injected model was used (Table 4). In general, we find that **the effect of pre-training bias is present, but minimal after fine-tuning**.

RQ3: Query Complexity. We defined a mathematical query complexity score (eq (9)) and compared it to model `EX` on queries in Spider4SSC separated by the hardness level proposed by Yu et al. [15]. Our results show a **clear proportionality between query complexity and model performance**. As query complexity increases, execution accuracy drops (see Fig. 5). Most importantly, we find **performance crossover between SQL and Cypher**. in Table 5a, SQL is the clear winner for easy queries (58% `EX`), but Cypher's performance is more robust on both "medium" (38% vs 34% `EX`) and "hard" (14% vs 6% `EX`) queries. This expands our RQ1 findings, showing that SQL is not universally better. While **SQL performs better for basic questions, Cypher's syntax is better at expressing complex questions**.

RQ4: Schema Quality. We defined two schema types with different level of detail, No Range summarises object (table) and property (column) names, while Full Compact also includes data types and relationships (foreign keys). From results in Table 6a we observed SPARQL and SQL performs best with Full Compact schema. However, in case of Cypher, No Range schema wins (38.82% vs 35.20%). We hypothesised that this was caused by token truncation and calculated input truncation statistics (see Appendix E). Table E.10 shows that the Full Compact Cypher schema has 14.5% mean truncation ratio, while No Range only has 7.7%. Thus, we conducted ablation study of model context length (Section 4.3.6). Table 8 shows that increasing model input context length to 1024 tokens, the Full Compact schema's performance improved from 35.20% to 37.83%, implying that performance was being bottlenecked by context length. When increasing to 2048 token model, where truncation is minimal for both No Range and Full Compact schema (only 0.18% and 0.52% resp.), the No Range schema (40.63% `EX`) still

outperformed the Full Compact schema (39.47% EX). This suggests that the extra information in Full Compact (i.e., data types and relationships, as seen in Appendix B) acts as information noise that slightly confuses the model. Thus, we conclude that **for SQL and SPARQL, the optimal schema is Full Compact, while Cypher benefits more from the shorter No Range representation**.

RQ5: Multi-task Learning. Finally, we asked if a single jointly-fine-tuned model could generalise better than single-query-language models. For each schema type, we jointly trained one model on all three query languages in Spider4SSC and evaluated them individually in each language. Table 7 shows that **multi-query-language learning does not universally improve performance**. However, we see effects when looking at each language separately. SQL improved by avg. +2.47% EX across all schema types, suggesting that **exposure to graph structures improves reasoning in the relational database model**. On the other hand, graph languages were harmed by the joint-fine-tuning, with Cypher degrading by -2.08% and SPARQL's by -0.38%. This suggests that **forcing a single model to learn both relational and graph query generation creates interference that degrades graph performance**.

6. Future Work

Most importantly, we want to appeal to further research in text-to-query modelling and general query language research to not be limited to just single query languages only. Our results show that it is important to study the tradeoffs between query languages and data models with respect to the given task. The **choice of query language and data model** should not be fixed from the start, it **should be another parameter to evaluate by experiments**. We are missing a field of applications where different choices of query language and data model could affect performance dramatically.

Secondly, although our experiments covered SQL, SPARQL, and Cypher and investigated their relative performance, we still largely fine-tuned them as separate systems. An interesting future path is to **explore shared representations or controlled mappings that can bridge query languages while preserving semantic equivalence**. While we entertained this idea in RQ5, more sophisticated methods and evaluation are needed.

Thirdly, investigation into other query languages and data models such as document oriented, object oriented or other graph-based systems, will

also extend the generalisability of our findings. The context length of the model is another limitation that should be studied in more detail. Future research should focus on strategies for handling longer schema input in small models or selective schema encoding that reduces truncation without loss of information.

Although the correlations between query complexity and execution accuracy are statistically significant, further validation is required. A robust approach would be to compute confidence intervals over execution outcomes for each query across clean, dirty, and injected runs and test whether these intervals overlap. This would confirm whether the observed differences represent consistent effects rather than random variation.

7. Acknowledgements

I would like to express my deep gratitude for the diligent supervision of Yasutaka Fujimoto and the unyielding support of all the members and resources of Fujimoto laboratory at Yokohama National University. Namely, I would like to thank my fellow Katiene Brice Clena Traore for thorough brainstorming sessions on the paper contents. In addition, a large thank you to Zahyra Valdez Rodrigues for her consultations and endless emotional support. Finally, thank you to John Ebot Besong and Awungabeh Flavis Akawung for wisdom, advice and academical support during the full lifecycle of conceptualising and writing this paper.

8. Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

9. CRediT authorship contribution statement

Martin Vejvar. Writing – original draft, Writing – review & editing, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization

Yasutaka Fujimoto. Writing – review & editing, Supervision, Resources, Funding acquisition

10. Declaration of generative AI and AI-assisted technologies in the manuscript preparation process.

During the preparation of this work the author used OpenAI GPT-5 and Google Gemini 2.5-flash and 2.5-pro to consolidate his understanding of related work, general brainstorming of ideas, reviewing the content of conclusions and double checking our writing style. After using this tool/service, we reviewed and edited the content as needed and take full responsibility for the content of the published article.

11. Data Availability

Fine-tuning data are available to download at [60]. Filtered Pre-training data is available in two parts at [80] and [81].

Appendix A. Query Complexity Factors

Given a query q in language $\mathcal{L}$, we define a set of structural complexity factors that are measured via language-adapted rules. All complexity factors are functions $f_X^{\mathcal{L}}(q)$ counting particular syntactic elements of type X.

Appendix A.1. Distinct Variables

The number of unique variables or identifiers:

$$f_{\text{distinct}}^{\mathcal{L}}(q) = |\{v \mid v \in \mathcal{V}^{\mathcal{L}}(q)\}| \tag{A.1}$$

where $\mathcal{V}^{\mathcal{L}}(q)$ is extracted as follows:

- **SPARQL**: all terms of the form `?var`
- **SQL**: column names qualified by table or alias (e.g. `table.column`, `alias.column`)
- **Cypher**: node or relationship variables in patterns (e.g. identifiers in `(a:Label)`, `[r:TYPE]`)

Appendix A.2. Tokens

The total number of lexical tokens in the query:

$$f_{\text{tokens}}(q) = |\mathcal{T}(q)| \tag{A.2}$$

where $\mathcal{T}(q)$ is defined as the set of non-whitespace substrings of q (i.e. splitting q at whitespace).

Appendix A.3. Keywords

The number of reserved language keywords present:

$$f^{\mathcal{L}}_{\text{keywords}}(q) = |\{t \in \mathcal{T}(q) \mid t \in K^{\mathcal{L}}\}| \tag{A.3}$$

where $K^{\mathcal{L}}$ is the set of reserved keywords for language $\mathcal{L}$, matched case-insensitively (see tab. A.9 for all keywords).

Table A.9: List of unique keywords used to count the $f^{\mathcal{L}}_{keywords}$ factor for each language.

Language	Keywords
SPARQL	BASE, PREFIX, SELECT, DISTINCT, REDUCED, CONSTRUCT, DESCRIBE, ASK, WHERE, FROM, NAMED, GRAPH, GROUP, BY, HAVING, ORDER, ASC, DESC, LIMIT, OFFSET, VALUES, FILTER, OPTIONAL, UNION, MINUS, EXISTS, NOT EXISTS, BIND, BINDINGS, SERVICE, isIRI, isURI, isBLANK, isLITERAL, isNUMERIC, BOUND, SAMETERM, STR, LANG, LANGMATCHES, DATATYPE, sameTerm, langMatches, regex, COUNT, SUM, AVG, MIN, MAX, SAMPLE, GROUP_CONCAT
SQL	ABORT, ACTION, ADD, AFTER, ALL, ALTER, ALWAYS, ANALYZE, AND, AS, ASC, ATTACH, AUTOINCREMENT, BEFORE, BEGIN, BETWEEN, BY, CASCADE, CASE, CAST, CHECK, COLLATE, COLUMN, COMMIT, CONFLICT, CONSTRAINT, CREATE, CROSS, CURRENT, CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, DATABASE, DEFAULT, DEFERRABLE, DEFERRED, DELETE, DESC, DETACH, DISTINCT, DO, DROP, EACH, ELSE, END, ESCAPE, EXCEPT, EXCLUDE, EXCLUSIVE, EXISTS, EXPLAIN, FAIL, FILTER, FIRST, FOLLOWING, FOR, FOREIGN, FROM, FULL, GENERATED, GLOB, GROUP, GROUPS, HAVING, IF, IGNORE, IMMEDIATE, IN, INDEX, INDEXED, INITIALLY, INNER, INSERT, INSTEAD, INTERSECT, INTO, IS, ISNULL, JOIN, KEY, LAST, LEFT, LIKE, LIMIT, MATCH, MATERIALIZED, NATURAL, NO, NOT, NOTHING, NOTNULL, NULL, NULLS, OF, OFFSET, ON, OR, ORDER, OTHERS, OUTER, OVER, PARTITION, PLAN, PRAGMA, PRECEDING, PRIMARY, QUERY, RAISE, RANGE, RECURSIVE, REFERENCES, REGEXP, REINDEX, RELEASE, RENAME, REPLACE, RESTRICT, RETURNING, RIGHT, ROLLBACK, ROW, ROWS, SAVEPOINT, SELECT, SET, TABLE, TEMP, TEMPORARY, THEN, TIES, TO, TRANSACTION, TRIGGER, UNBOUNDED, UNION, UNIQUE, UPDATE, USING, VACUUM, VALUES, VIEW, VIRTUAL, WHEN, WHERE, WINDOW, WITH, WITHOUT
Cypher	ACCESS, ACTIVE, ADMIN, ADMINISTRATOR, ALIAS, ALIASES, ALL, ALL_SHORTEST_PATHS, ALTER, AND, ANY, ARRAY, AS, ASC, ASCENDING, ASSIGN, AT, AUTH, BINDINGS, BOOL, BOOLEAN, BOOSTED, BOTH, BREAK, BUILT, BY, CALL, CASCADE, CASE, CHANGE, CIDR, COLLECT, COMMAND, COMMANDS, COMPOSITE, CONCURRENT, CONSTRAINT, CONSTRAINTS, CONTAINS, CONTINUE, COPY, COUNT, CREATE, CSV, CURRENT, DATA, DATABASE, DATABASES, DATE, DATETIME, DBMS, DEALLOCATE, DEFAULT, DEFINED, DELETE, DENY, DESC, DESCENDING, DESTROY, DETACH, DIFFERENT, DISTINCT, DRIVER, DROP, DRYRUN, DUMP, DURATION, EACH, EDGE, ELEMENT, ELEMENTS, ELSE, ENABLE, ENCRYPTED, END, ENDS, ERROR, EXECUTABLE, EXECUTE, EXIST, EXISTENCE, EXISTS, FAIL, FALSE, FIELDTERMINATOR, FINISH, FLOAT, FOR, FOREACH, FROM, FULLTEXT, FUNCTION, FUNCTIONS, GRANT, GRAPH, GRAPHS, GROUP, GROUPS, HEADERS, HOME, ID, IF, IMMUTABLE, IMPERSONATE, IN, INDEX, INDEXES, INF, INFINITY, INSERT, INT, INTEGER, IS, JOIN, KEY, LABEL, LABELS, LEADING, LIMITROWS, LIST, LOAD, LOCAL, LOOKUP, MANAGEMENT, MAP, MATCH, MERGE, NAME, NAMES, NAN, NEW, NFC, NFD, NFKC, NFKD, NODE, NODES, NODETACH, NONE, normalisE, normalisED, NOT, NOTHING, NOWAIT, NULL, OF, OFFSET, ON, ONLY, OPTION, OPTIONAL, OPTIONS, OR, ORDER, PASSWORD, PASSWORDS, PATH, PATHS, PLAINTEXT, POINT, POPULATED, PRIMARIES, PRIMARY, PRIVILEGE, PRIVILEGES, PROCEDURE, PROCEDURES, PROPERTIES, PROPERTY, PROVIDER, PROVIDERS, RANGE, READ, REALLOCATE, REDUCE, REL, RELATIONSHIP, RELATIONSHIPS, REMOVE, RENAME, REPEATABLE, REPLACE, REPORT, REQUIRE, REQUIRED, RESTRICT, RETURN, REVOKE, ROLE, ROLES, ROW, ROWS, SCAN, SEC, SECOND, SECONDARIES, SECONDARY, SECONDS, SEEK, SERVER, SERVERS, SET, SETTING, SETTINGS, SHORTEST, SHORTEST_PATH, SHOW, SIGNED, SINGLE, SKIPROWS, START, STARTS, STATUS, STOP, STRING, SUPPORTED, SUSPENDED, TARGET, TERMINATE, TEXT, THEN, TIME, TIMESTAMP, TIMEZONE, TO, TOPOLOGY, TRAILING, TRANSACTION, TRANSACTIONS, TRAVERSE, TRIM, TRUE, TYPE, TYPED, TYPES, UNION, UNIQUE, UNIQUENESS, UNWIND, URL, USE, USER, USERS, USING, VALUE, VARCHAR, VECTOR, VERTEX, WAIT, WHEN, WHERE, WITH, WITHOUT, WRITE, XOR, YIELD, ZONE, ZONED

Appendix A.4. Joins/Traversals

The join and traversal complexity is quantified by counting explicit and implicit connection points between tables, triples or graph patterns within SQL, SPARQL, and Cypher queries respectively:

$$f^{\mathcal{L}}_{\text{joins}}(q) = |\mathcal{J}_{\mathcal{L}}(q)| \tag{A.4}$$

where $\mathcal{J}_{\mathcal{L}}(q)$ is the set of relevant join or traversal points, defined as:

***SQL**: Locating JOIN keywords and comma-separated tables in FROM clause.*

$$\mathcal{J}_{\text{SQL}}(q) = \mathcal{J}_{\text{exp}}(q) \cup \mathcal{J}_{\text{imp}}(q) \tag{A.5}$$

where

$$\mathcal{J}_{\text{exp}}(q) = \{\, j \mid j \text{ is an explicit JOIN keyword in } q \,\}$$
$$\mathcal{J}_{\text{imp}}(q) = \bigcup_{C \in \mathcal{F}(q)} \{\, t_i \mid 1 \le i \le |C| - 1 \,\}$$

$\mathcal{F}(q)$ is the set of comma-separated table lists in each FROM clause of q that are not accompanied by explicit JOINs (to support SQL-89 syntax for implicit Cartesian JOIN operation).

***SPARQL**: Expanding compact triple notation and counting bridges.*

$$\mathcal{J}_{\text{SPARQL}}(q) = \{(v, t_i, t_j) : v \in \mathcal{V}_{SO}(q),\ t_i \neq t_j,\ v \in t_i,\ v \in t_j\} \tag{A.6}$$

where $\mathcal{V}_{SO}(q)$ is the set of variables appearing as subject or object in every expanded triple pattern of q.

***Cypher**: Counting traversals between nodes.*

$$\mathcal{J}_{\text{CYPHER}}(q) = \{\, e \mid e \text{ is an explicit traversal pattern in } q \,\} \tag{A.7}$$

where an explicit traversal pattern is any edge pattern in `MATCH` clauses, such as `()-[]->()`, `()<-[]-()` or `()-[]-()`.

Appendix A.5. Nesting Depth

Nesting depth complexity factor represents the structural depth of the query. In other words, it quantizes the depth of nested subqueries across SQL, SPARQL, and Cypher. The nesting depth complexity is given by

$$f^{\mathcal{L}}_{\text{nesting}}(q) = \mathcal{D}(q) \tag{A.8}$$

where $\mathcal{D}(q)$ is the maximum level of nested subqueries or scoped blocks present within query q, defined for each language as follows:

***SQL**: Counting nested `SELECT` queries within parentheses.*

$$\mathcal{D}_{\text{SQL}}(q) = \max_{p \in \mathcal{P}(q)} \text{nesting}(p) \tag{A.9}$$

where $\mathcal{P}(q)$ is the set of subquery positions identified by the pattern `"(SELECT"`, and nesting(p) counts the nesting levels by matching pairs of parentheses.

***SPARQL**: Counting nested `SELECT` queries within braces.*

$$\mathcal{D}_{\text{SPARQL}}(q) = \max_{b \in \mathcal{B}(q)} \text{nesting}(b) \tag{A.10}$$

where $\mathcal{B}(q)$ is the set of positions where a nested `SELECT` appears after an opening brace `"{SELECT"`, and the nesting depth is measured by matching pairs of braces.

***Cypher**: Counting nested `CALL` or `MATCH` scopes within braces.*

$$\mathcal{D}_{\text{CYPHER}}(q) = \max_{c \in \mathcal{C}(q)} \text{nesting}(c) \tag{A.11}$$

where $\mathcal{C}(q)$ denotes positions of explicit nested scopes introduced either by `"CALL {"` or `"{MATCH"`, and nesting levels are determined by counting matching pairs of braces.

Appendix A.6. Filters

The number of explicit filtering conditions:

$$f^{\mathcal{L}}_{\text{filters}}(q) = \begin{cases} |\{\texttt{FILTER} \text{ occurrences in } q\}| & \text{SPARQL} \\ |\{\texttt{WHERE}, \texttt{CASE}, \texttt{IF} \text{ in } q\}| & \text{SQL} \\ |\{\texttt{WHERE} \text{ occurrences in } q\}| & \text{Cypher} \end{cases}$$

Appendix A.7. Aggregations

The number of explicit aggregation expressions:

$$f^{\mathcal{L}}_{\text{aggregations}}(q) = \begin{cases} |\{\texttt{GROUP BY}, \texttt{AGGREGATE}, \texttt{DISTINCT} \text{ in } q\}| & \text{SPARQL} \\ |\{\texttt{GROUP BY}, \texttt{DISTINCT} \text{ in } q\}| & \text{SQL} \\ |\{\texttt{DISTINCT} \text{ in } q\}| & \text{Cypher} \end{cases}$$

Appendix A.8. Sorting and Limiting

The number of result sorting or limiting constructs (`ORDER BY`, `LIMIT`, `SKIP`, etc.):

$$f^{\mathcal{L}}_{\text{sorting}}(q) = |\{s \mid s \text{ is a sort/limit statement in } q\}| \tag{A.12}$$

Appendix A.9. Projections

The number of explicit projection clauses (selecting output fields):

$$f^{\mathcal{L}}_{\text{projections}}(q) = |\{p \mid p \text{ is a projection operation in } q\}| \tag{A.13}$$

where p is `SELECT` for SQL and SPARQL and `RETURN` for Cypher.

Appendix B. Schema serialisation Examples

Below are complete examples of the two schema representation variants (*No Range* and *Full Compact*) for SQL, SPARQL, and Cypher. Each example shows the full serialisation structure "$(\mathcal{L})\ q \mid D_{nm} \mid \mathcal{S}$" as defined in eq. (8).

Appendix B.1. No Range

Listing 3: No Range serialisation for SQL.

```
(sql) How many singers do we have? | concert_singer |
stadium : stadium_id, location, name, capacity, highest, lowest,
    ↪ average |
singer : singer_id, name, country, song_name, song_release_year,
    ↪ age, is_male |
concert : concert_id, concert_name, theme, stadium_id, year |
singer_in_concert : concert_id, singer_id
```

Listing 4: No Range serialisation for SPARQL.

```
(sparql) How many singers do we have? | concert_singer | concert:
    ↪  year, theme, stadium_id, concert_name, concert_id,
    ↪ STADIUM_ID | singer_in_concert: singer_id, concert_id,
    ↪ SINGER_ID, CONCERT_ID | singer: song_release_year,
    ↪ song_name, singer_id, name, is_male, country, age |
    ↪ stadium: stadium_id, name, lowest, location, highest,
    ↪ capacity, average
```

Listing 5: No Range serialisation for Cypher.

```
(cypher) How many singers do we have? | concert_singer |
    ↪ ROOT__singer_in_concert: uri,
    ↪ singer_in_concert__concert_id,
    ↪ singer_in_concert__singer_id,
    ↪ singer_in_concert__CONCERT_ID,
    ↪ singer_in_concert__SINGER_ID | ROOT__concert: uri,
    ↪ concert__stadium_id, concert__year, concert__concert_name,
    ↪  concert__concert_id, concert__theme, concert__STADIUM_ID
```

```
↪ | ROOT__singer: uri, singer__age, singer__country,
↪ singer__singer_id, singer__song_name, singer__name,
↪ singer__is_male, singer__song_release_year | ROOT__stadium
↪ : uri, stadium__location, stadium__lowest,
↪ stadium__stadium_id, stadium__highest, stadium__capacity,
↪ stadium__average, stadium__name
```

Appendix B.2. Full Compact

Listing 6: Full Compact serialisation for SQL.

```
(sql) How many singers do we have? | concert_singer | stadium :
↪ stadium_id (number), location (text), name (text),
↪ capacity (number), highest (number), lowest (number),
↪ average (number) | singer : singer_id (number), name (text
↪ ), country (text), song_name (text), song_release_year (
↪ text), age (number), is_male (others) | concert :
↪ concert_id (number), concert_name (text), theme (text),
↪ stadium_id (stadium_id), year (text) | singer_in_concert :
↪  concert_id (concert_id), singer_id (singer_id)
```

Listing 7: Full Compact serialisation for SPARQL.

```
(sparql) How many singers do we have? | concert_singer | concert:
↪  year (string), theme (string), stadium_id (integer),
↪ concert_name (string), concert_id (integer), STADIUM_ID (
↪ stadium) | singer_in_concert: singer_id (integer),
↪ concert_id (integer), SINGER_ID (singer), CONCERT_ID (
↪ concert) | singer: song_release_year (string), song_name (
↪ string), singer_id (integer), name (string), is_male (
↪ boolean), country (string), age (integer) | stadium:
↪ stadium_id (integer), name (string), lowest (integer),
↪ location (string), highest (integer), capacity (integer),
↪ average (integer)
```

Listing 8: Full Compact serialisation for Cypher.

```
(cypher) How many singers do we have? | concert_singer |
↪ ROOT__singer_in_concert: uri (String),
↪ singer_in_concert__concert_id (Long),
↪ singer_in_concert__singer_id (Long),
↪ singer_in_concert__CONCERT_ID (ROOT__concert),
↪ singer_in_concert__SINGER_ID (ROOT__singer) |
↪ ROOT__concert: uri (String), concert__stadium_id (Long),
↪ concert__year (String), concert__concert_name (String),
↪ concert__concert_id (Long), concert__theme (String),
```

```
↪ concert__STADIUM_ID (ROOT__stadium) | ROOT__singer: uri (
↪ String), singer__age (Long), singer__country (String),
↪ singer__singer_id (Long), singer__song_name (String),
↪ singer__name (String), singer__is_male (Boolean),
↪ singer__song_release_year (String) | ROOT__stadium: uri (
↪ String), stadium__location (String), stadium__lowest (Long
↪ ), stadium__stadium_id (Long), stadium__highest (Long),
↪ stadium__capacity (Long), stadium__average (Long),
↪ stadium__name (String)
```

Appendix C. Filtered Datasets

Listing 9: Main code for filtering out text with potential Query lanugage chunks.

```
def filter_and_annotate(examples, tokenizer=None):
    texts = examples["text"]
    tokenized = tokenizer(texts, add_special_tokens=False,
        ↪ truncation=False)
    num_tokens = [len(ids) for ids in tokenized["input_ids"]]
    filters_triggered_list = []
    ql_type_list = []
    matched_substring_list = []
    keyword_counts_list = []
    for text in texts:
        filters_triggered = []
        ql_type = None
        matched_substring = None
        tokens_upper = re.findall(r'\w+', text.upper())
        token_counts = Counter(tokens_upper)
        keyword_counts = {kw: token_counts[kw] for kw in
            ↪ query_keywords if token_counts[kw] > 0}
        if keyword_counts:
            filters_triggered.append("keyword")
        for pattern, lang in QL_PATTERNS:
            match = re.search(pattern, text, re.IGNORECASE)
            if match:
                filters_triggered.append("regex")
                ql_type = lang
                matched_substring = match.group(0)
                break
        filters_triggered_list.append(filters_triggered)
        ql_type_list.append(ql_type or "")
        matched_substring_list.append(matched_substring or "")
        keyword_counts_list.append(json.dumps(keyword_counts))
```

```
return {
    "filters_triggered": filters_triggered_list,
    "ql_type": ql_type_list,
    "matched_substring": matched_substring_list,
    "keyword_counts": keyword_counts_list,
    "num_tokens": num_tokens,
}
```

Appendix D. Prediction examples

Below, Cypher prediction does not take into account the concert_year parameter is part of the concert object and not the singer object.

Listing 10: Sample prediction from text-to-Cypher fine-tuned uT5-clean model.

```
{
    "prediction": "match (T2:ROOT__singer_in_concert)-[:
        ↪ singer_in_concert__SINGER_ID]->(T1:ROOT__singer) where
        ↪  T2.singer_in_concert__year = 2014 return T1.
        ↪ singer__name as t1_name",
    "lang": "cypher",
    "query": "MATCH (t2:ROOT__singer_in_concert)-[:
        ↪ singer_in_concert__SINGER_ID]->(t1:ROOT__singer) MATCH
        ↪  (t2:ROOT__singer_in_concert)-[:
        ↪ singer_in_concert__CONCERT_ID]->(t3:ROOT__concert)
        ↪ WHERE t3.concert__year = '2014' RETURN t1.singer__name
        ↪  AS t1_name",
    "question": "What are the names of the singers who performed
        ↪ in a concert in 2014?",
    "context": "(cypher) What are the names of the singers who
        ↪ performed in a concert in 2014? | concert_singer |
        ↪ ROOT__singer: uri (String), singer__age (Long),
        ↪ singer__country (String), singer__singer_id (Long),
        ↪ singer__song_name (String), singer__name (String),
        ↪ singer__is_male (Boolean), singer__song_release_year (
        ↪ String) | ROOT__concert: uri (String),
        ↪ concert__stadium_id (Long), concert__year (String),
        ↪ concert__concert_name (String), concert__concert_id (
        ↪ Long), concert__theme (String), concert__STADIUM_ID (
        ↪ ROOT__stadium) | ROOT__singer_in_concert: uri (String)
        ↪ , singer_in_concert__concert_id (Long),
        ↪ singer_in_concert__singer_id (Long),
        ↪ singer_in_concert__CONCERT_ID (ROOT__concert),
        ↪ singer_in_concert__SINGER_ID (ROOT__singer) |
        ↪ ROOT__stadium: uri (String), stadium__location (String
```

```
        ↪ ), stadium__lowest (Long), stadium__stadium_id (Long),
        ↪  stadium__highest (Long), stadium__capacity (Long),
        ↪ stadium__average (Long), stadium__name (String)",
    "label": "match (t2:ROOT__singer_in_concert)-[:
        ↪ singer_in_concert__SINGER_ID]->(t1:ROOT__singer) match
        ↪  (t2:ROOT__singer_in_concert)-[:
        ↪ singer_in_concert__CONCERT_ID]->(t3:ROOT__concert)
        ↪ where t3.concert__year = '2014' return t1.singer__name
        ↪  as t1_name",
    "db_id": "concert_singer",
    "db_path": "/home/vejvar-martin-nj/git/picard/.cache/
        ↪ downloads/extracted/
        ↪ c702c18c8d855b7bc0a53f5b230cd5314a83d607fea4df3ad5612a557fae3dd2
        ↪ /Spider4SSC/database",
    "db_table_names": [],
    "db_column_names": [],
    "db_foreign_keys": []
},
```

Appendix E. T5 Input token truncation

As mentioned in Section 4.3.4, our models can encounter context length issues due to the default maximum input length of 512 tokens in the T5 model. Whenever the tokenised input exceeded this limit, the remainder was truncated, resulting in information loss from the serialised schema. This effect is particularly pronounced for **Cypher**, where node and relationship names tend to be significantly longer than in SQL or SPARQL. By contrast, SQL schemas are generally more compact, while SPARQL lies between the two.

Listing 11: Sample question from department_store DB in Spider4SSC train set.

```
What is the document id, template id and description for
    document named 'Robbin CV'?
```

Example token counts. Given the question in Listing 11 from the Spider4SSC training set, the tokenised input lengths with the full compact schema using the T5-base tokeniser[12] are:

- SQL: 695 tokens (see Listing 12)

[12] `https://huggingface.co/google-t5/t5-base`

- SPARQL: 816 tokens (see Listing 13)
- Cypher: 1568 tokens (see Listing 14)

The Cypher input exceeds the T5 context limit ca three times, leading to significant information loss.

Schema-level analysis. The truncation varies depending on the serialisation of the schema. Table E.10 reports the average token counts and truncation ratio in all 4525 samples of Spider4SSC.

Table E.10: Full statistics of token count across the 4525 samples of Spider4SSC and truncation ratio for for maximum input length of 512 tokens.

		Token count		**Truncation ratio (max 512 tokens)**		**Number of samples longer than**		
Language	Schema	**mean**	**std**	**mean**	**std**	**512 tokens**	**1024 tokens**	**2048 tokens**
cypher	Full Compact	575	511	14.52%	23.03%	1623	609	58
	No Range	402	354	7.71%	16.17%	1110	124	58
	No Schema	28	6	0.00%	0.00%	0	0	0
sparql	Full Compact	326	282	4.39%	12.12%	859	58	14
	No Range	213	177	0.92%	6.57%	146	25	0
	No Schema	28	6	0.00%	0.00%	0	0	0
sql	Full Compact	295	256	3.18%	10.60%	522	58	14
	No Range	189	156	0.63%	5.31%	124	14	0
	No Schema	21	6	0.00%	0.00%	0	0	0

Appendix E.1. Examples of the Full Compact schema serialisation

Listing 12: Full compact schema serialisation for SQL.

```
department_store | addresses : address_id (number) ,
    ↪ address_details (text) | staff : staff_id (number) ,
    ↪ staff_gender (text) , staff_name (text) | suppliers :
    ↪ supplier_id (number) , supplier_name (text) ,
    ↪ supplier_phone (text) | department_store_chain :
    ↪ dept_store_chain_id (number) , dept_store_chain_name (text
    ↪ ) | customers : customer_id (number) , payment_method_code
    ↪  (text) , customer_code (text) , customer_name (text) ,
    ↪ customer_address (text) , customer_phone (text) ,
    ↪ customer_email (text) | products : product_id (number) ,
    ↪ product_type_code (text) , product_name (text) ,
    ↪ product_price (number) | supplier_addresses : supplier_id
    ↪ (supplier_id) , address_id (address_id) , date_from (time)
    ↪  , date_to (time) | customer_addresses : customer_id (
    ↪ customer_id) , address_id (address_id) , date_from (time)
    ↪ , date_to (time) | customer_orders : order_id (number) ,
    ↪ customer_id (customer_id) , order_status_code (text) ,
```

```
    ↪ order_date (time) | department_stores : dept_store_id (
    ↪ number) , dept_store_chain_id (dept_store_chain_id) ,
    ↪ store_name (text) , store_address (text) , store_phone (
    ↪ text) , store_email (text) | departments : department_id (
    ↪ number) , dept_store_id (dept_store_id) , department_name
    ↪ (text) | order_items : order_item_id (number) , order_id (
    ↪ order_id) , product_id (product_id) | product_suppliers :
    ↪ product_id (product_id) , supplier_id (supplier_id) ,
    ↪ date_supplied_from (time) , date_supplied_to (time) ,
    ↪ total_amount_purchased (text) , total_value_purchased (
    ↪ number) | staff_department_assignments : staff_id (
    ↪ staff_id) , department_id (department_id) ,
    ↪ date_assigned_from (time) , job_title_code (text) ,
    ↪ date_assigned_to (time)
```

Listing 13: Full compact schema serialisation for SPARQL.

```
department_store | customer_orders: order_status_code (string) ,
    ↪ order_id (integer) , order_date (dateTime) , customer_id (
    ↪ integer) , CUSTOMER_ID (customers) |
    ↪ department_store_chain: dept_store_chain_name (string) ,
    ↪ dept_store_chain_id (integer) | departments: dept_store_id
    ↪  (integer) , department_name (string) , department_id (
    ↪ integer) , DEPT_STORE_ID (department_stores) |
    ↪ staff_department_assignments: staff_id (integer) ,
    ↪ job_title_code (string) , department_id (integer) ,
    ↪ date_assigned_to (dateTime) , date_assigned_from (dateTime
    ↪ ) , STAFF_ID (staff) , DEPARTMENT_ID (departments) |
    ↪ supplier_addresses: supplier_id (integer) , date_to (
    ↪ dateTime) , date_from (dateTime) , address_id (integer) ,
    ↪ SUPPLIER_ID (suppliers) , ADDRESS_ID (addresses) |
    ↪ suppliers: supplier_phone (string) , supplier_name (string
    ↪ ) , supplier_id (integer) | department_stores: store_phone
    ↪  (string) , store_name (string) , store_email (string) ,
    ↪ store_address (string) , dept_store_id (integer) ,
    ↪ dept_store_chain_id (integer) , DEPT_STORE_CHAIN_ID (
    ↪ department_store_chain) |
staff: staff_name (string) , staff_id (integer) , staff_gender (
    ↪ string) | customers: payment_method_code (string) ,
    ↪ customer_phone (string) , customer_name (string) ,
    ↪ customer_id (integer) , customer_email (string) ,
    ↪ customer_code (string) , customer_address (string) |
    ↪ customer_addresses: date_to (dateTime) , date_from (
    ↪ dateTime) , customer_id (integer) , address_id (integer) ,
    ↪  CUSTOMER_ID (customers) , ADDRESS_ID (addresses) |
    ↪ product_suppliers: total_value_purchased (decimal) ,
```

```
    ↪ total_amount_purchased (string) , supplier_id (integer) ,
    ↪ product_id (integer) , date_supplied_to (dateTime) ,
    ↪ date_supplied_from (dateTime) , SUPPLIER_ID (suppliers) ,
    ↪ PRODUCT_ID (products) | products: product_type_code (
    ↪ string) , product_price (decimal) , product_name (string)
    ↪ , product_id (integer) | order_items: product_id (integer)
    ↪  , order_item_id (integer) , order_id (integer) ,
    ↪ PRODUCT_ID (products) , ORDER_ID (customer_orders) |
    ↪ addresses: address_id (integer) , address_details (string)
```

Listing 14: Full compact schema serialisation for Cypher.

```
department_store | ROOT__addresses: uri (String) ,
    ↪ addresses__address_id (Long) , addresses__address_details
    ↪ (String) | ROOT__department_stores: uri (String) ,
    ↪ department_stores__store_name (String) ,
    ↪ department_stores__store_address (String) ,
    ↪ department_stores__store_email (String) ,
    ↪ department_stores__dept_store_id (Long) ,
    ↪ department_stores__dept_store_chain_id (Long) ,
    ↪ department_stores__store_phone (String) ,
    ↪ department_stores__DEPT_STORE_CHAIN_ID
    ↪ (ROOT__department_store_chain) | ROOT__customers: uri
    ↪ (String) , customers__customer_address (String) ,
    ↪ customers__payment_method_code (String) ,
    ↪ customers__customer_email (String) ,
    ↪ customers__customer_name (String) ,
    ↪ customers__customer_code (String) ,
    ↪ customers__customer_phone (String) ,
    ↪ customers__customer_id (Long) | ROOT__suppliers: uri
    ↪ (String) , suppliers__supplier_id (Long) ,
    ↪ suppliers__supplier_name (String) ,
    ↪ suppliers__supplier_phone (String) | ROOT__departments:
    ↪ uri (String) , departments__dept_store_id (Long) ,
    ↪ departments__department_id (Long) ,
    ↪ departments__department_name (String) ,
    ↪ departments__DEPT_STORE_ID (ROOT__department_stores) |
ROOT__staff: uri (String) , staff__staff_gender (String) ,
    ↪ staff__staff_id (Long) , staff__staff_name (String) |
    ↪ ROOT__order_items: uri (String) , order_items__product_id
    ↪ (Long) , order_items__order_id (Long) ,
    ↪ order_items__order_item_id (Long) , order_items__ORDER_ID
    ↪ (ROOT__customer_orders) , order_items__PRODUCT_ID
    ↪ (ROOT__products) | ROOT__product_suppliers: uri (String)
    ↪ , product_suppliers__date_supplied_from (LocalDateTime) ,
    ↪ product_suppliers__date_supplied_to (LocalDateTime) ,
```

```
↪ product_suppliers__product_id (Long) ,
↪ product_suppliers__total_value_purchased (Double) ,
↪ product_suppliers__total_amount_purchased (String) ,
↪ product_suppliers__supplier_id (Long) ,
↪ product_suppliers__PRODUCT_ID (ROOT__products) ,
↪ product_suppliers__SUPPLIER_ID (ROOT__suppliers) |
↪ ROOT__supplier_addresses: uri (String) ,
↪ supplier_addresses__date_from (LocalDateTime) ,
↪ supplier_addresses__address_id (Long) ,
↪ supplier_addresses__date_to (LocalDateTime) ,
↪ supplier_addresses__supplier_id (Long) ,
↪ supplier_addresses__ADDRESS_ID (ROOT__addresses) ,
↪ supplier_addresses__SUPPLIER_ID (ROOT__suppliers) |
↪ ROOT__customer_orders: uri (String) ,
↪ customer_orders__order_status_code (String) ,
↪ customer_orders__order_date (LocalDateTime) ,
↪ customer_orders__customer_id (Long) ,
↪ customer_orders__order_id (Long) ,
↪ customer_orders__CUSTOMER_ID (ROOT__customers) |
↪ ROOT__department_store_chain: uri (String) ,
↪ department_store_chain__dept_store_chain_name (String) ,
↪ department_store_chain__dept_store_chain_id (Long) |
↪ ROOT__staff_department_assignments: uri (String) ,
↪ staff_department_assignments__department_id (Long) ,
↪ staff_department_assignments__job_title_code (String) ,
↪ staff_department_assignments__date_assigned_from
↪ (LocalDateTime) ,
↪ staff_department_assignments__date_assigned_to
↪ (LocalDateTime) , staff_department_assignments__staff_id
↪ (Long) , staff_department_assignments__DEPARTMENT_ID
↪ (ROOT__departments) ,
↪ staff_department_assignments__STAFF_ID (ROOT__staff) |
↪ ROOT__products: uri (String) , products__product_id
↪ (Long) , products__product_name (String) ,
↪ products__product_price (Double) ,
↪ products__product_type_code (String) |
↪ ROOT__customer_addresses: uri (String) ,
↪ customer_addresses__customer_id (Long) ,
↪ customer_addresses__date_from (LocalDateTime) ,
↪ customer_addresses__address_id (Long) ,
↪ customer_addresses__date_to (LocalDateTime) ,
↪ customer_addresses__ADDRESS_ID (ROOT__addresses) ,
↪ customer_addresses__CUSTOMER_ID (ROOT__customers)
```

Appendix F. Pseudocode

Appendix F.1. Multi-Language Evaluation Logic

The following pseudocode demonstrates our extension of the evaluation loop, highlighting the new asynchronous execution strategy and result normalisation required to support SPARQL and Cypher alongside SQL.

Listing 15: Pseudocode for the unified execution and normalisation pipeline.

```
# standardised Types and Rounding
DATATYPES = {
    "xsd:integer": int, "xsd:decimal": float,
    "xsd:boolean": bool, "xsd:dateTime": str
    }
ROUND_DIGITS = 1

def postprocess_syntax(query: str) -> str:
    """Normalise␣syntactic␣artifacts␣introduced␣by␣the␣generation
       ␣model␣(e.g.,␣T5␣tokenizer␣splits)␣prior␣to␣execution."""
    # 1. Remove decoder-specific control tokens
    # (Common artifacts in sequence-to-sequence generation)
    query = query.replace("</s>", "")
    query = query.replace("<pad>", "")

    # 2. Collapse duplicated braces
    # (Fixes common JSON/Map formatting errors in Cypher
       generation)
    query = query.replace("{{", "{")
    query = query.replace("}}", "}")

    # 3. Repair split operators
    # (Recombines operators split by tokenization, e.g., "> =" ->
       ">=")
    query = query.replace(">␣=", ">=")
    query = query.replace("<␣=", "<=")
    query = query.replace("!␣=", "!=")

    # 4. Standard whitespace cleanup
    return query.strip()

def normalise_value(val):
    """Unify␣types␣across␣SQL,␣RDF,␣and␣Graph␣outputs"""
    if isinstance(val, float):
        return round(val, ROUND_DIGITS)
    if isinstance(val, RDF_Literal):
```

```
            py_type = DATATYPES.get(val.datatype, str)
            return py_type(val.value)
        if val is None or val == "null":
            return None
        return val

def normalise_results(rows: List[Tuple]) -> List[Tuple]:
    """Apply normalisation to every cell in the result set"""
    return [tuple(normalise_value(c) for c in row) for row in
        rows]

async def eval_exec_match(db_path, pred_str, gold_str, lang):
    # 1. Pre-process Query Strings
    pred_str = postprocess_syntax(pred_str) # Fix "> =" etc.
    if not KEEP_DISTINCT:
        pred_str = remove_distinct(pred_str, lang)
        gold_str = remove_distinct(gold_str, lang)

    ```
    # 2. Asynchronous Execution (Concurrent)
    # We run both queries concurrently to handle network latency
    # for remote RDF/Graph endpoints.
    try:
        gold_res, pred_res = await asyncio.gather(
            exec_driver(db_path, gold_str, lang),
            exec_driver(db_path, pred_str, lang)
        )
    except ExecutionError:
        return 0 # Fail if execution crashes

    # 3. normalise Data Types
    # Converts XSD/Java types to standard Python primitives
    g_rows = normalise_results(gold_res)
    p_rows = normalise_results(pred_res)

    # 4. Compare using Standard Bag Semantics (2020, Zhong et al
        .)
    # (Checks for column permutations and row set equality)
    order_matters = "order by" in gold_str.lower()
    return result_eq(g_rows, p_rows, order_matters)
```

The result_eq is kept as is in the original implementation by Zhong et al. [77].
```

References

[1] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, Journal of Machine Learning Research 21 (2020) 1–67.

[2] Y. Nan, B. Ding, K. Chakrabarti, Enhancing text-to-sql capabilities of large language models, arXiv preprint arXiv:2305.06983 (2023).

[3] M. Besta, R. Gerstenberger, E. Peter, M. Fischer, M. Podstawski, C. Barthels, G. Alonso, T. Hoefler, Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries, ACM Computing Surveys 56 (9 2023). doi:10.1145/3604932.
URL https://dl.acm.org/doi/10.1145/3604932

[4] S. Coulondre, Cg-sql: a front-end language for conceptual graph knowledge bases, Knowledge-Based Systems 12 (5) (1999) 293–302. doi:https://doi.org/10.1016/S0950-7051(99)00021-0.
URL https://www.sciencedirect.com/science/article/pii/S0950705199000210

[5] R. Cyganiak, D. Wood, M. Lanthaler, G. Klyne, J. J. Carroll, B. McBride, Rdf 1.1 concepts and abstract syntax, W3C recommendation 25 (02) (2014) 1–22.

[6] G. Szárnyas, J. Marton, J. Maginecz, D. Varró, Reducing property graph queries to relational algebra for incremental view maintenance, arXiv preprint arXiv:1806.07344 (2018).

[7] D. Lukovnikov, A. Fischer, J. Lehmann, S. Auer, Neural network-based question answering over knowledge graphs on word and character level, in: Proceedings of the 26th International Conference on World Wide Web, WWW '17, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 2017, p. 1211–1220. doi:10.1145/3038912.3052675.
URL https://doi.org/10.1145/3038912.3052675

[8] X. Bi, H. Nie, X. Zhang, X. Zhao, Y. Yuan, G. Wang, Unrestricted multi-hop reasoning network for interpretable question answering

over knowledge graph, Knowledge-Based Systems 243 (2022) 108515. doi:https://doi.org/10.1016/j.knosys.2022.108515.
URL https://www.sciencedirect.com/science/article/pii/S0950705122002222

[9] M. Mountantonakis, Y. Tzitzikas, Using multiple rdf knowledge graphs for enriching chatgpt responses, in: G. De Francisci Morales, C. Perlich, N. Ruchansky, N. Kourtellis, E. Baralis, F. Bonchi (Eds.), Machine Learning and Knowledge Discovery in Databases: Applied Data Science and Demo Track, Springer Nature Switzerland, Cham, 2023, pp. 324–329.

[10] G. Futia, A. Vetro, A. Melandri, J. C. D. Martin, Training neural language models with sparql queries for semi-automatic semantic mapping, Procedia Computer Science 137 (2018) 187–198. doi:10.1016/J.PROCS.2018.09.018.

[11] G. Ambrosio-Cestero, J. R. Ruiz-Sarmiento, J. Gonzalez-Jimenez, The robot@home2 dataset: A new release with improved usability tools, SoftwareX 23 (2023) 101490. doi:10.1016/J.SOFTX.2023.101490.

[12] S. Sivasubramaniam, C. Osei-Akoto, Y. Zhang, K. Stockinger, J. Fürst, Sm3-text-to-query: Synthetic multi-model medical text-to-query benchmark, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems, Vol. 37, Curran Associates, Inc., 2024, pp. 88627–88663.
URL https://proceedings.neurips.cc/paper_files/paper/2024/file/a182a8e6ebc91728b6e6b6382c9f7b1e-Paper-Datasets_and_Benchmarks_Track.pdf

[13] M. Vejvar, balanced-plms: Code for pre-training balanced language models, gitHub repository. Accessed: 2025-11-14 (2025).
URL https://github.com/vejvarm/balanced-plms

[14] M. Vejvar, ut5-ssc: Code for fine-tuning unbiased t5 models on spider4ssc, gitHub repository. Accessed: 2025-11-14 (2025).
URL https://github.com/vejvarm/uT5-ssc

[15] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-

labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: E. Riloff, D. Chiang, J. Hockenmaier, J. Tsujii (Eds.), Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 3911–3921. doi:10.18653/v1/D18-1425.
URL https://aclanthology.org/D18-1425/

[16] K. Xu, L. Wu, Z. Wang, Y. Feng, V. Sheinin, SQL-to-text generation with graph-to-sequence model, in: E. Riloff, D. Chiang, J. Hockenmaier, J. Tsujii (Eds.), Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 931–936. doi:10.18653/v1/D18-1112.
URL https://aclanthology.org/D18-1112/

[17] Q. Lyu, K. Chakrabarti, S. Hathi, S. Kundu, J. Zhang, Z. Chen, Hybrid ranking network for text-to-sql (2020). arXiv:2008.04759.
URL https://arxiv.org/abs/2008.04759

[18] X. V. Lin, R. Socher, C. Xiong, Bridging textual and tabular data for cross-domain text-to-sql semantic parsing, Findings of the Association for Computational Linguistics Findings of ACL: EMNLP 2020 (2020) 4870–4888doi:10.18653/V1/2020.FINDINGS-EMNLP.438.

[19] T. Scholak, N. Schucher, D. Bahdanau, Picard: Parsing incrementally for constrained auto-regressive decoding from language models, EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings (2021) 9895–9901doi:10.18653/V1/2021.EMNLP-MAIN.779.

[20] Y. Gan, X. Chen, J. Xie, M. Purver, J. R. Woodward, J. Drake, Q. Zhang, Natural sql: Making sql easier to infer from natural language specifications, Findings of the Association for Computational Linguistics, Findings of ACL: EMNLP 2021 (2021) 2030–2042doi:10.18653/V1/2021.FINDINGS-EMNLP.174.

[21] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, K. Yu, Lgesql: Line graph enhanced text-to-sql model with mixed local and non-local relations, ACL-IJCNLP 2021 - 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on

Natural Language Processing, Proceedings of the Conference 1 (2021) 2541–2555. doi:10.18653/v1/2021.acl-long.198.

[22] Y. Nan, H. Nori, Y. Tang, K. Chakrabarti, Ehrsql: A benchmark dataset for text-to-sql tasks with electronic health records, arXiv preprint arXiv:2208.13629 (2022).

[23] M. J. Al-Muhammed, D. W. Lonsdale, Ontology-aware dynamically adaptable free-form natural language agent interface for querying databases, Knowledge-Based Systems 239 (2022) 108012. doi:10.1016/J.KNOSYS.2021.108012.

[24] B. Hui, R. Geng, L. Wang, B. Qin, Y. Li, B. Li, J. Sun, Y. Li, S2sql: Injecting syntax to question-schema interaction graph encoder for text-to-sql parsers, Proceedings of the Annual Meeting of the Association for Computational Linguistics (2022) 1254–1262doi:10.18653/V1/2022.FINDINGS-ACL.99.

[25] C. Gao, B. Li, W. Zhang, W. Lam, B. Li, F. Huang, L. Si, Y. Li, Towards generalizable and robust text-to-SQL parsing, in: Y. Goldberg, Z. Kozareva, Y. Zhang (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2022, Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2022, pp. 2113–2125. doi:10.18653/v1/2022.findings-emnlp.155.
URL https://aclanthology.org/2022.findings-emnlp.155/

[26] H. Li, J. Zhang, C. Li, H. Chen, Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql, Proceedings of the 37th AAAI Conference on Artificial Intelligence, AAAI 2023 37 (2023) 13067–13075. doi:10.1609/AAAI.V37I11.26535.

[27] G. Jeong, M. Han, S. Kim, Y. Lee, J. Lee, S. Park, H. Kim, Improving text-to-sql with a hybrid decoding method, Entropy 25 (3 2023). doi:10.3390/E25030513.

[28] M. Han, S. Park, S. Kim, H. Kim, Bridging the gap between text-to-sql research and real-world applications: A unified all-in-one framework for text-to-sql, Knowledge-Based Systems 306 (2024) 112697. doi:10.1016/J.KNOSYS.2024.112697.

[29] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls, Advances in Neural Information Processing Systems 36 (2024).

[30] M. Pourreza, D. Rafiei, Dts-sql: Decomposed text-to-sql with small large language models, EMNLP 2024 - 2024 Conference on Empirical Methods in Natural Language Processing, Findings of EMNLP 2024 (2024) 8212–8220doi:10.18653/V1/2024.FINDINGS-EMNLP.481.

[31] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-sql empowered by large language models: A benchmark evaluation, Proceedings of the VLDB Endowment 17 (2024) 1132–1145. doi:10.14778/3641204.3641221.

[32] J. Yang, B. Hui, M. Yang, J. Yang, J. Lin, C. Zhou, Synthesizing text-to-sql data from weak and strong llms, Proceedings of the Annual Meeting of the Association for Computational Linguistics 1 (2024) 7864–7875. doi:10.18653/V1/2024.ACL-LONG.425.

[33] Z. Zhang, Y. Chen, C. Guo, J. Song, G. He, Enhancing text-to-sql generation with language sequential consistency, Neurocomputing 658 (2025) 131721. doi:10.1016/J.NEUCOM.2025.131721.

[34] M. Bhardwaj, H. Ethari, D. S. Moirangthem, Ezsql: An sql intermediate representation for improving sql-to-text generation, Expert Systems with Applications 289 (2025) 128293. doi:10.1016/J.ESWA.2025.128293.

[35] M. Azmy, P. Shi, J. Lehmann, A.-C. Ngonga Ngomo, Lc-quad: A corpus for complex question answering over knowledge graphs, in: Proceedings of the 17th International Semantic Web Conference (ISWC 2018), 2018, pp. 210–218.

[36] D. Chaves-Fraga, F. Priyatna, A. Cimmino, J. Toledo, E. Ruckhaus, O. Corcho, Gtfs-madrid-bench: A benchmark for virtual knowledge graph access in the transport domain, Journal of Web Semantics 65 (2020) 100596. doi:https://doi.org/10.1016/j.websem.2020.100596.
URL https://www.sciencedirect.com/science/article/pii/S1570826820300354

[37] A. Panchbhai, T. Soru, E. Marx, Exploring sequence-to-sequence models for sparql pattern composition, Communications in Computer and Information Science 1232 (2020) 158–165. doi:10.1007/978-3-030-65384-2_12.

[38] X. Ye, S. Yavuz, K. Hashimoto, Y. Zhou, C. Xiong, Rng-kbqa: Generation augmented iterative ranking for knowledge base question answering, Proceedings of the Annual Meeting of the Association for Computational Linguistics 1 (2022) 6032–6043. doi:10.18653/V1/2022.ACL-LONG.417.

[39] M. R. A. H. Rony, U. Kumar, R. Teucher, L. Kovriguina, J. Lehmann, Sgpt: A generative approach for sparql query generation from natural language questions, IEEE Access 10 (2022) 70712–70723. doi:10.1109/ACCESS.2022.3188714.

[40] Y. Tan, D. Min, Y. Li, W. Li, N. Hu, Y. Chen, G. Qi, Can chatgpt replace traditional kbqa models? an in-depth analysis of the question answering performance of the gpt llm family, Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 14265 LNCS (2023) 348–367. doi:10.1007/978-3-031-47240-4_19.

[41] M. A. Borroto, F. Ricca, Sparql-qa-v2 system for knowledge base question answering, Expert Systems with Applications 229 (2023) 120383. doi:10.1016/J.ESWA.2023.120383.

[42] J. Qi, C. Su, Z. Guo, L. Wu, Z. Shen, L. Fu, X. Wang, C. Zhou, Enhancing sparql query generation for knowledge base question answering systems by learning to correct triplets, Applied Sciences (Switzerland) 14 (2 2024). doi:10.3390/APP14041521.

[43] J. Lee, H. Shin, Sparkle: Enhancing sparql generation with direct kg integration in decoding, Expert Systems with Applications 289 (2025) 128263. doi:10.1016/J.ESWA.2025.128263.

[44] G. Feng, G. Zhu, S. Shi, Y. Sun, Z. Fan, S. Gao, J. Hu, Robust nl-to-cypher translation for kbqa: Harnessing large language model with chain of prompts, Communications in Computer and Information Science 1923 CCIS (2023) 317–326. doi:10.1007/978-981-99-7224-1_25.

[45] M. Hornsteiner, M. Kreussel, C. Steindl, F. Ebner, P. Empl, S. Schönig, Real-time text-to-cypher query generation with large language models for graph databases, Future Internet 16 (12 2024). doi:10.3390/FI16120438.

[46] Y. Zou, Y. Wang, D. Liu, Q2cypher: Converting natural language questions to cypher with fine-tuned large language models, 2024 5th International Conference on Artificial Intelligence and Computer Engineering, ICAICE 2024 (2024) 783–788doi:10.1109/ICAICE63571.2024.10863984.

[47] Q. B. H. Tran, A. A. Waheed, S. T. Chung, Robust text-to-cypher using combination of bert, graphsage, and transformer (cobgt) model, Applied Sciences (Switzerland) 14 (9 2024). doi:10.3390/APP14177881.

[48] S. Soleymani, N. Gravel, K. Kochut, N. Kannan, Task splitting and prompt engineering for cypher query generation in domain-specific knowledge graphs, bioRxiv (2025). arXiv:https://www.biorxiv.org/content/early/2025/04/24/2025.04.23.649790.full.pdf, doi:10.1101/2025.04.23.649790.
URL https://www.biorxiv.org/content/early/2025/04/24/2025.04.23.649790

[49] Y. Wan, Z. Chen, Y. Liu, C. Chen, M. Packianather, Prompting large language models based on semantic schema for text-to-cypher transformation towards domain q&a, Decision Support Systems 199 (2025) 114553. doi:https://doi.org/10.1016/j.dss.2025.114553.
URL https://www.sciencedirect.com/science/article/pii/S016792362500154X

[50] C. Yang, C. Li, X. Hu, H. Yu, J. Lu, Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline with large language models, Information Processing & Management 63 (2026) 104280. doi:10.1016/J.IPM.2025.104280.

[51] P. Wang, T. Shi, C. K. Reddy, Text-to-sql generation for question answering on electronic medical records, in: Proceedings of The Web Conference 2020, WWW '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 350–361. doi:10.1145/3366423.3380120.
URL https://doi.org/10.1145/3366423.3380120

[52] J. Park, Y. Cho, H. Lee, J. Choo, E. Choi, Knowledge graph-based question answering with electronic health records, in: K. Jung, S. Yeung, M. Sendak, M. Sjoding, R. Ranganath (Eds.), Proceedings of the 6th Machine Learning for Healthcare Conference, Vol. 149 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 36–53.
URL https://proceedings.mlr.press/v149/park21a.html

[53] C. Kosten, P. Cudré-Mauroux, K. Stockinger, Spider4sparql: A complex benchmark for evaluating knowledge graph question answering systems, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 5272–5281.
URL https://arxiv.org/abs/2309.16248v2

[54] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, P. Huang, Towards robustness of text-to-SQL models against synonym substitution, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Association for Computational Linguistics, Online, 2021, pp. 2505–2515. doi:10.18653/v1/2021.acl-long.195.
URL https://aclanthology.org/2021.acl-long.195/

[55] Y. Gan, X. Chen, M. Purver, Exploring underexplored limitations of cross-domain text-to-sql generalization, in: EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings, Association for Computational Linguistics (ACL), 2021, pp. 8926–8931. doi:10.18653/V1/2021.EMNLP-MAIN.702.
URL https://aclanthology.org/2021.emnlp-main.702/

[56] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, M. Richardson, Structure-grounded pretraining for text-to-sql, NAACL-HLT 2021 - 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference (2021) 1337–1350doi:10.18653/V1/2021.NAACL-MAIN.105.
URL https://aclanthology.org/2021.naacl-main.105/

[57] J. Qi, J. Tang, Z. He, X. Wan, Y. Cheng, C. Zhou, X. Wang, Q. Zhang, Z. Lin, Rasat: Integrating relational structures into pretrained seq2seq

model for text-to-sql, Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022 (2022) 3215–3229doi:10.18653/V1/2022.EMNLP-MAIN.211.

[58] D. Calvanese, B. Cogrel, S. Komla-Ebri, R. Kontchakov, D. Lanti, M. Rezk, M. Rodriguez-Muro, G. Xiao, Ontop: Answering sparql queries over relational databases, Semantic Web 8 (3) (2017) 471–487.

[59] M. Vejvar, Y. Fujimoto, Spider4ssc & s2clite: A text-to-multi-query-language dataset using lightweight ontology-agnostic sparql to cypher parser (2025). arXiv:2511.09354.
URL https://arxiv.org/abs/2511.09354

[60] M. Vejvar, Y. Fujimoto, Spider4ssc - a semantically equivalent text-to-sql/sparql/cypher dataset, Mendeley Data, V2 (2025). doi:10.17632/4dnvv92trn.2.
URL https://data.mendeley.com/datasets/4dnvv92trn/2

[61] A. K. Chandra, P. M. Merlin, Optimal implementation of conjunctive queries in relational data bases, in: Proceedings of the Ninth Annual ACM Symposium on Theory of Computing, STOC '77, Association for Computing Machinery, New York, NY, USA, 1977, p. 77–90. doi:10.1145/800105.803397.
URL https://doi.org/10.1145/800105.803397

[62] A. Chandra, D. Harel, Structure and complexity of relational queries, Journal of Computer and System Sciences 25 (1) (1982) 99–128. doi:https://doi.org/10.1016/0022-0000(82)90012-5.
URL https://www.sciencedirect.com/science/article/pii/0022000082900125

[63] J. Pérez, M. Arenas, C. Gutierrez, Semantics and complexity of sparql, in: International semantic web conference, Springer, 2006, pp. 30–43.

[64] M. Y. Vardi, The complexity of relational query languages (extended abstract), in: Proceedings of the Fourteenth Annual ACM Symposium on Theory of Computing, STOC '82, Association for Computing Machinery, New York, NY, USA, 1982, p. 137–146. doi:10.1145/800070.802186.
URL https://doi.org/10.1145/800070.802186

[65] C. H. Papadimitriou, M. Yannakakis, On the complexity of database queries, Journal of Computer and System Sciences 58 (3) (1999) 407–427. doi:https://doi.org/10.1006/jcss.1999.1626.
URL https://www.sciencedirect.com/science/article/pii/S0022000099916264

[66] J. G. March, H. A. Simon, Organizations, John wiley & sons, 1993.

[67] D. J. Campbell, Task complexity: A review and analysis, Academy of management review 13 (1) (1988) 40–52.

[68] A. F. Borthick, P. L. Bowen, D. R. Jones, M. H. K. Tse, The effects of information request ambiguity and construct incongruence on query development, Decision Support Systems 32 (1) (2001) 3–25.

[69] P. L. Bowen, R. A. O'Farrell, F. H. Rohde, An empirical investigation of end-user query development, Information Systems Research 20 (2009) 565–584. doi:10.1287/ISRE.1080.0181.
URL https://dl.acm.org/doi/10.1287/isre.1080.0181

[70] A. Suhr, M.-W. Chang, P. Shaw, K. Lee, Exploring unexplored generalization challenges for cross-database semantic parsing, in: D. Jurafsky, J. Chai, N. Schluter, J. Tetreault (Eds.), Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 8372–8388. doi:10.18653/v1/2020.acl-main.742.
URL https://aclanthology.org/2020.acl-main.742/

[71] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, et al., Xiyan-sql: A multi-generator ensemble framework for text-to-sql, arXiv preprint arXiv:2411.08599 (2024).

[72] L.-P. Meyer, J. Frey, F. Brei, N. Arndt, Assessing sparql capabilities of large language models, arXiv preprint arXiv:2409.05925 (2024).

[73] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, A. Green, J. Hidders, B. Li, L. Libkin, V. Marsault, W. Martens, et al., Pg-schema: Schemas for property graphs, Proceedings of the ACM on Management of Data 1 (2) (2023) 1–25.

[74] J. Barrasa, A. Cowley, neosemantics (n10s): Neo4j rdf semantics toolkit-neo4j labs (2021).

[75] sparkle-g, Sparql 1.1 grammar pretty printer and syntactic vallidator (2013).
URL https://code.google.com/archive/p/sparkle-g/

[76] S. Sivasubramaniam, C. E. Osei-Akoto, Y. Zhang, K. Stockinger, J. Fuerst, dataset_analysis.ipynb, part of the SM3-Text-to-Query project, commit 357d11f (2024).
URL https://github.com/jf87/SM3-Text-to-Query/blob/357d11fc041df7c415a20e358561a4e8e03e2523/src/evaluation/dataset_analysis.ipynb

[77] R. Zhong, T. Yu, D. Klein, Semantic evaluation for text-to-SQL with distilled test suites, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online, 2020, pp. 396–411. doi:10.18653/v1/2020.emnlp-main.29.
URL https://aclanthology.org/2020.emnlp-main.29/

[78] A. Gokaslan, V. Cohen, Openwebtext corpus, http://Skylion007.github.io/OpenWebTextCorpus (2019).

[79] C. Stadler, M. Saleem, Q. Mehmood, C. Buil-Aranda, M. Dumontier, A. Hogan, A. C. N. Ngomo, Lsq 2.0: A linked dataset of sparql query logs, Semantic Web 15 (2024) 167–189.

[80] M. Vejvar, Y. Fujimoto, Query-free openwebtext - part 1: Clean and dirty corpus, Mendeley Data, V1 (2025). doi:10.17632/m2gdxppkvj.1.
URL https://data.mendeley.com/datasets/m2gdxppkvj/1

[81] M. Vejvar, Y. Fujimoto, Spider4ssc - a semantically equivalent text-to-sql/sparql/cypher dataset, Mendeley Data, V1 (2025). doi:10.17632/f46xx3jmg3.1.
URL https://data.mendeley.com/datasets/f46xx3jmg3/1